AOSP 进阶丛书 · 第四册 · ANDROID 13 (API 33)

AMS 四大组件调度源码精读

Activity 冷启动十步、ClientTransaction 生命周期事务化、Service/Broadcast/Provider 同构骨架与欠条-兑现模型、computeOomAdj 优先级记账机器与 ANR 体系——逐段注释读懂调度中枢。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：从"会用"到"会管"——调度中枢的解剖课 2

第一章 Activity 启动：十步冷启动全链路 2

第二章 ClientTransaction：生命周期事务化 9

第三章 Service：最安静的组件 11

第四章 Broadcast：系统的脉搏 14

第五章 ContentProvider：数据的跨进程门户 15

第六章 进程优先级：computeOomAdj 记账机器 16

第七章 深水区：ART 内存执行系统与 lmkd 21

第八章 线程发动机：Handler/Looper/MessageQueue 全解 22

第九章 设计决策八问 24

第十章 实战：读一次真实启动的 trace 25

第十一章 Service 完整生命周期：从 startService 到销毁的全本账 26

第十二章 Broadcast 深水区：动态注册桥、有序队列与超时机器 35

第十三章 ContentProvider 发布-等待-回收：三条链条的合流 41

第十四章 updateOomAdjLSP 内部：一次全量核算的流水线与空进程回收 50

附录 A 观测实操 56

附录 B 自检题 56

附录 C 崩溃/卡顿与调度的对照表 56

附录 D 快问快答（面试扫雷 20 条） 57

教科书习题集（分级） 58

术语表（按拼音排序） 61

排障手册：AMS 与四大组件调度问题定位 62

本册速览

Activity 冷启动十步（含 realStart/performLaunch 逐段注释）
ClientTransaction 事务化：callbacks+终态，cycleToPath 补路径
四组件同构骨架：欠条-兑现 · ANR 五源 · computeOomAdj 记账机器
深水区：ART CC/MTERP/lmkd/冻结 · 线程发动机 Handler→epoll
三补全：PendingIntent/ActivityResult/ReceiverDispatcher

前言：从"会用"到"会管"——调度中枢的解剖课

前三册走完了开机与通信两条主线，本册进入系统开发者的日常主战场：**AMS（ActivityManagerService）及其分管的四大组件调度**。你将看到 startActivity 背后的每一次跨进程往返、Service/Broadcast/Provider 三兄弟的同构骨架、进程优先级如何被一台"记账机器"实时核算，以及 ANR 从诞生到弹窗的全过程。

本册的代码讲解采用**逐段注释**体例：每段源码摘录都带中文行内注释（标注为"讲解注释"）+ 代码后的"逐段精读"编号解读，初学者可以逐行跟读。源码基线不变（本树 Android 13；注意本树 `ActivityStackSupervisor` 更名为 `ActivityTaskSupervisor`，写作时两处均有注明）。

第一章 Activity 启动：十步冷启动全链路

设计洞察 · 学习目标

1. 不看图数出冷启动十步并标注每步所在进程与跳转方式（Binder/socket）；2. 说清 executeRequest 三段式守门与 realStartActivityLocked 事务组装的分工；3. 解释 attachApplicationLocked 为何是四组件的"总兑现"时刻；4. 用 am start -W 的 TotalTime 把启动耗时切成孵化/事务/首帧三段

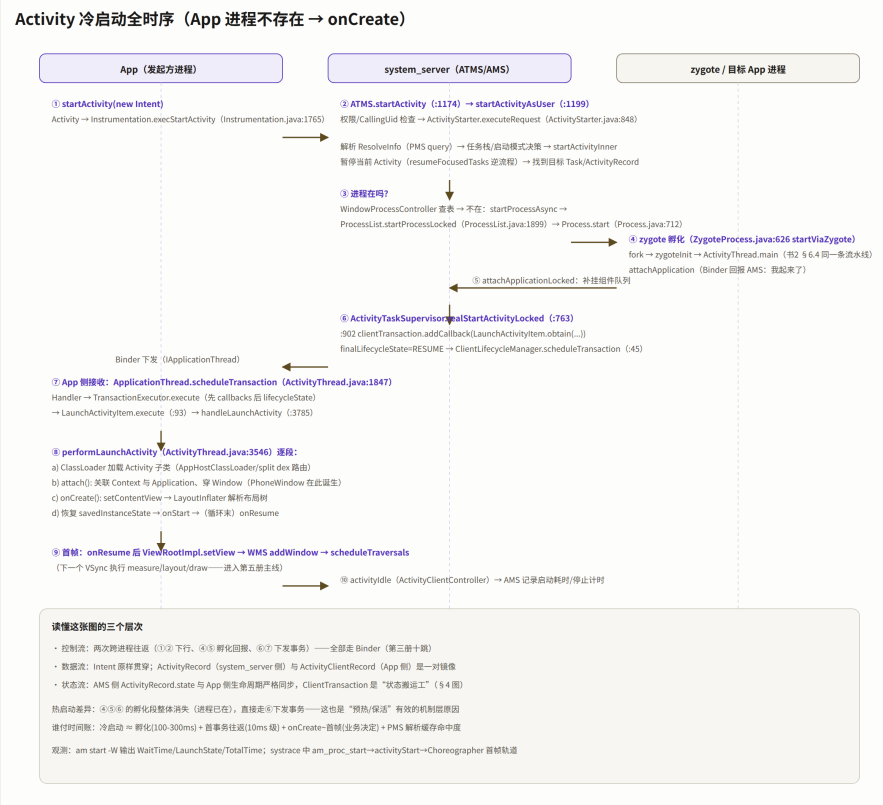


图 1 Activity 冷启动全时序：发起方 App、system_server、zygote/目标进程三泳道，十步逐跳

1.1 全链路鸟瞰

冷启动十步（图①~⑩）：`startActivity` → `Instrumentation.execStartActivity`（`Instrumentation.java:1765`）→ `ATMS.startActivity`（`ActivityTaskManagerService.java:1174`）→ `ActivityStarter.executeRequest`（`ActivityStarter.java:848`）→ 进程不存在则孵化（`ProcessList.startProcessLocked:1899` → zygote，书2 同款流水线）→ `attachApplication` 回报 → `ActivityTaskSupervisor.realStartActivityLocked`（`:763`）组装事务 → Binder 下发 → App 侧 `LaunchActivityItem.execute`（`:93`）→ `handleLaunchActivity`（`ActivityThread.java:3785`）→ `performLaunchActivity`（`:3546`）。

1.2 executeRequest：守门与决策（逐段注释）

java frameworks/.../wm/ActivityStarter.java:848 起（节选+讲解注释）

```
private int executeRequest(Request request) {
// ── 第 1 段：身份与快速失败 ──────────────────────────
// 讲解注释：所有跨进来的请求先"验明正身"。callingUid 来自 Binder（书3 §5.2），
// startAnyPerm 拥有者可以绕过组件导出检查（系统特权应用如 shell）。
...
// ── 第 2 段：解析目标 ──────────────────────────────
// 讲解注释：queryIntentActivities 走 PMS（第六册）的组件倒排索引，
// 拿到 ActivityInfo；没有匹配直接返回 START_INTENT_NOT_RESOLVED。
...
// ── 第 3 段：权限闸门 ──────────────────────────────
// 讲解注释：startActivity 权限、跨用户检查、抽象 URI 授权（UriGrantsManager，
// SystemServer Bootstrap 批先于 AMS 启动的意义在此）。
...
// ── 第 4 段：交给内层 ──────────────────────────────
// 讲解注释：走 startActivityInner 做任务栈决策（launchMode/flags/任务亲和性），
// 决定"复用哪个 Task、放在哪、是否清栈"。
}
```

先修要求与学习路线

先修：已读前三册（架构/启动/Binder）。

读完本书你将能够：

- 追踪AMS 与四大组件调度的核心机制的完整调用链
- 解释关键设计决策的"为什么"
- 使用 dumpsys 定位本领域问题
- 回答相关面试题（书7 对应章节）

> **设计洞察 · 逐段精读：executeRequest 的三重身份**
> 第 1 段是**安检员**（身份可信，其余免谈）；第 2 段是**翻译官**（把语义化的 Intent 翻译成确定的组件与类名）；第 3、4 段才是**调度员**。这个"先验证、再解析、后决策"的三段式是所有系统服务入口方法的通用骨架——与你在第三册看到的 servicemanager.addService 五道关卡同构。

1.3 realStartActivityLocked：组装事务（真实源码逐段注释）

java frameworks/.../wm/ActivityTaskSupervisor.java:895-933（节选，注释为讲解注释）

```
// Create activity launch transaction.
// 讲解注释：ClientTransaction.obtain 从对象池取事务；proc.getThread() 就是
// 目标进程的 IApplicationThread Binder 代理——事务的"收件人"。
final ClientTransaction clientTransaction = ClientTransaction.obtain(
proc.getThread(), r.token);
final boolean isTransitionForward = r.isTransitionForward();
final IBinder fragmentToken = r.getTaskFragment().getFragmentToken();
// 讲解注释：LaunchActivityItem 装的是"开工全套材料"：Intent、ActivityInfo、
// 全局/覆盖 Configuration（屏幕旋转等）、savedState（进程被杀后的现场恢复）、
// 结果与新 Intent、ProfilerInfo……App 侧 onCreate 需要的一切都在包里。
clientTransaction.addCallback(LaunchActivityItem.obtain(new Intent(r.intent),
System.identityHashCode(r), r.info,
mergedConfiguration.getGlobalConfiguration(),
mergedConfiguration.getOverrideConfiguration(), r.compat,
r.getFilteredReferrer(r.launchedFromPackage), task.voiceInteractor,
proc.getReportedProcState(), r.getSavedState(), r.getPersistentSavedState(),
results, newIntents, r.takeOptions(), isTransitionForward,
proc.createProfilerInfoIfNeeded(), r.assistToken, activityClientController,
r.shareableActivityToken, r.getLaunchedFromBubble(), fragmentToken));
// Set desired final state.
// 讲解注释：终态二选一——正常路径 ResumeActivityItem；先不显示（如启动到后台）
// 则 PauseActivityItem。App 侧会沿状态机补齐中间态（onCreate→onStart→onResume）。
final ActivityLifecycleItem lifecycleItem;
if (andResume) {
lifecycleItem = ResumeActivityItem.obtain(isTransitionForward);
} else {
lifecycleItem = PauseActivityItem.obtain();
}
clientTransaction.setLifecycleStateRequest(lifecycleItem);
// Schedule transaction.
// 讲解注释：ClientLifecycleManager.scheduleTransaction（:45）→ Binder 下发，
// 书3 十跳的又一次具体旅行。
mService.getLifecycleManager().scheduleTransaction(clientTransaction);
```

逐段精读三要点：①**事务的收件人是进程不是 Activity**——token（`r.token`）才是 Activity 的镜像句柄，同一进程的多个 Activity 共享一条 Binder 通道；②**"材料"与"终态"分离**——callbacks 描述"发生什么"，lifecycleState 描述"最终到哪"；③**andResume 分支**解释了一个常见现象：通知栏点进来的 Activity 可能走到 onCreate 但不 onResume（启动到后台）。

1.4 performLaunchActivity：App 侧的出生仪式（真实源码逐段注释）

java frameworks/base/core/java/android/app/ActivityThread.java:3546-3605（节选）

```
private Activity performLaunchActivity(ActivityClientRecord r, Intent customIntent) {
ActivityInfo aInfo = r.activityInfo;
// 讲解注释①：LoadedApk——本 APK 的"资源+类加载器"上下文；
// CONTEXT_INCLUDE_CODE：把 APK 的 dex 挂进 ClassLoader（split dex 路由于此）。
if (r.packageInfo == null) {
r.packageInfo = getPackageInfo(aInfo.applicationInfo, r.compatInfo,
Context.CONTEXT_INCLUDE_CODE);
```

```
}
ComponentName component = r.intent.getComponent();
...
// 讲解注释②：activity-alias 场景——manifest 的 targetActivity 指向真实类。
if (r.activityInfo.targetActivity != null) {
component = new ComponentName(...targetActivity);
}
ContextImpl appContext = createBaseContextForActivity(r);
Activity activity = null;
try {
// 讲解注释③：反射创建 Activity 实例。你的无参构造函数就是在这里被调用的；
// 此刻它还没有 Context——所以构造函数里不能调用任何需要 Context 的 API。
java.lang.ClassLoader cl = appContext.getClassLoader();
activity = mInstrumentation.newActivity(
cl, component.getClassName(), r.intent);
...
} catch (Exception e) { ... throw new RuntimeException("Unable to instantiate activity ...");
}
try {
// 讲解注释④：确保 Application 存在（首启动时在此 onCreate）；
// makeApplicationInner(false,...) 表示非重载路径。
Application app = r.packageInfo.makeApplicationInner(false, mInstrumentation);
...
if (activity != null) {
// 讲解注释⑤：attach——注入 Context/Window（PhoneWindow 诞生地）、
// 关联 Application 与 Intent；随后就是 onCreate（setContentView 的家）。
...
}
...
```

逐段精读：③与⑤合起来解释了经典面试题"Activity 构造函数与 onCreate 里各能做什么"——构造时只有裸对象，attach 后才有 Context 体系。④说明 Application.onCreate 的真实触发时机：**首个组件启动路径上，而不是进程启动时**（attach 时若未创建则补）。

源码锚点 · 第一章锚点速查

Instrumentation.java:1765 execStartActivity / ActivityTaskManagerService.java:1174/1199 startActivity(AsUser)
ActivityStarter.java:848 executeRequest / ActivityTaskSupervisor.java:763 realStartActivityLocked、:902 LaunchActivityItem 组装
ActivityThread.java:1847 scheduleTransaction / :3785 handleLaunchActivity / :3546 performLaunchActivity

1.5 任务栈决策：launchMode 与 flags 的裁决现场

`startActivityInner`（`ActivityStarter.java:1646` 注释所指）是 launchMode 的"法庭"。判案顺序固定，初学者背下这个顺序就不会迷路：

text 任务栈裁决顺序（简化自 ActivityStarter/Task 处理逻辑）

```
1 FLAG_ACTIVITY_NEW_TASK? —是→ 找 taskAffinity 匹配的目标 Task（没有则新建）
2 目标 Task 定下后，在栈顶找"可复用"的实例：
• singleTop：栈顶同组件 → 走 onNewIntent（不新建）
• singleTask：栈内存在 → 清其上方全部实例 + onNewIntent
• singleInstance：独占一个 Task（此 Task 永远只有它）
3 FLAG_ACTIVITY_CLEAR_TOP：栈内存在 → 销毁其上实例（与 launchMode 组合出 8 种行为）
4 默认 standard：无条件 new 实例压栈
5 结果落账：Task 组织进 RootWindowContainer；ActivityRecord.setState(INITIALIZING)
```

设计洞察 · 逐段精读：为什么决策这么复杂

因为 Android 的"任务"是**用户视角的聚合**（最近任务卡片）而非进程视角——跨进程的 Activity 能进同一张卡片，靠 taskAffinity 声明归属。裁决顺序里 NEW_TASK 排第一，因为它决定"去哪个法庭"；launchMode 决定"能不能复用被告席"。排障口诀：**launchMode 看 Task 内，flags 看 Task 间**。

1.6 attachApplicationLocked：孵化后的"总兑现"时刻

新进程回报 attachApplication 后，AMS 在 attachApplicationLocked 里做一次**集中兑现**（本册最重要的"还款日"）：

java am/ActivityManagerService.java attachApplicationLocked（骨架+讲解注释）

```
private boolean attachApplicationLocked(...) {
// 讲解注释①：进程记账入库：pid→ProcessRecord 绑定、
// proc.setThread(thread) 记下 IApplicationThread 代理（此后才有"收件人"）。
// 讲解注释②：优先级初算：generateProcessForLockedUpActivities 等路径
// 视 pending 组件给初始 state（别让新进程一出生就被 lmkd 盯上）。
// 讲解注释③：兑现三大欠条：
// a) mPendingServices → realStartServiceLocked（第三章）
// b) mPendingBroadcast → processCurBroadcastLocked（第四章）
// c) mLaunchingProviders → publishContentProviders（第五章）
// 讲解注释④：再跑一次 OomAdjuster（状态变了，待遇重算）。
// 讲解注释⑤：stable/unstable provider 计数、backup agent 等收尾。
}
```

逐段精读：**为什么集中在这里？** 因为 IApplicationThread 就位是"一切下发的前提"——四组件的欠条都等这一个信号。把这个函数读熟，你同时复习了四条组件链的交汇点，这也是 AMS 面试的高频"枢纽题"。

1.7 十步冷启动完整代码走读（教科书级逐步分解）

这是全书最重要的一节——每步都给出真实源码+注释+进程标注。

第 ① 步：App 调用 startActivity

java App 进程 · 主线程

```
// Activity.java
@Override
public void startActivity(Intent intent) {
startActivityForResult(intent, -1); // -1 = 不需要返回结果
}
// Activity.java → Instrumentation.java:1765
```

```
public Instrumentation.ActivityResult execStartActivity(
Context who, IBinder contextThread, IBinder token, Activity target,
Intent intent, int requestCode, Bundle options) {
// 讲解注释①：包装 Intent → ActivityOptions
// 讲解注释②：获取 ATMS 代理（书3 §九 第①跳）
int result = ActivityTaskManager.getService()
.startActivity(whoThread, who.getBasePackageName(), intent,
intent.resolveTypeIfNeeded(who.getContentResolver()),
token, target != null ? target.mEmbeddedID : null,
requestCode, 0, null, options);
// 讲解注释③：检查结果码（START_SUCCESS=0 / START_INTENT_NOT_RESOLVED 等）
checkStartActivityResult(result, intent);
// ★ 此处发生第一次 Binder 跳（App → system_server）
}
```

第 ② 步：ATMS.startActivity

java system_server 进程 · binder 线程

```
// ActivityTaskManagerService.java:1174
public final int startActivity(IApplicationThread caller, String callingPackage,
Intent intent, String resolvedType, IBinder resultTo, String resultWho,
int requestCode, int startFlags, ProfilerInfo profilerInfo, Bundle bOptions) {
// 讲解注释④：权限检查（CallingUid 来自 Binder 驱动，不可伪造）
// 讲解注释⑤：转发到 startActivityAsUser → ActivityStarter
return startActivityAsUser(caller, callingPackage, intent, resolvedType,
resultTo, resultWho, requestCode, startFlags, profilerInfo, bOptions,
UserHandle.getCallingUserId());
}
```

第 ③ 步：ActivityStarter.executeRequest（守门与决策）

java system_server 进程

```
// ActivityStarter.java:848
private int executeRequest(Request request) {
// 讲解注释⑥：身份检查——拥有 START_ANY_ACTIVITY 权限可绕过导出检查
// 讲解注释⑦：PMS 查询——resolveActivity 找到目标 ActivityInfo
//  （走 Computer 快照，书6 §二）
// 讲解注释⑧：权限闸门——startActivity 权限 / 跨用户检查 / URI 授权
// 讲解注释⑨：任务栈决策——launchMode/flags/affinity → startActivityInner
//  （决定"复用哪个 Task、放在哪、是否清栈"）
}
```

第 ④ 步：进程不存在 → 孵化

java system_server 进程

```
// 如果目标进程不存在：
// ProcessList.java:1899 → Process.start(:712) → ZygoteProcess(:626)
// 讲解注释⑩：组装启动参数串（uid/gid/ABI/seInfo/rlimits/capabilities）
// 写 zygote socket（书2 §六 同一条流水线）
// ★ 此处走 socket（非 Binder）到 zygote 进程
```

第 ⑤ 步：attachApplication（进程回报）

java 新进程 → system_server · binder

```
// ActivityThread.main() → AMS.attachApplication() → attachApplicationLocked
// 讲解注释⑪：集中兑现三大欠条：
// a) mPendingServices → realStartServiceLocked
// b) mPendingBroadcast → processCurBroadcastLocked
// c) mLaunchingProviders → publishContentProviders
// 讲解注释⑫：再跑一次 OomAdjuster（新进程的初始优先级）
```

第 ⑥ 步：realStartActivityLocked（组装事务）

java system_server 进程

```
// ActivityTaskSupervisor.java:763
boolean realStartActivityLocked(ActivityRecord r, WindowProcessController proc,
boolean andResume, boolean checkConfig) throws TransactionException {
// 讲解注释⑬：组装 ClientTransaction（:902）
final ClientTransaction clientTransaction = ClientTransaction.obtain(
proc.getThread(), r.token);
clientTransaction.addCallback(LaunchActivityItem.obtain(
new Intent(r.intent), // Intent 原件
System.identityHashCode(r), // ident
r.info, // ActivityInfo
mergedConfiguration, // 全局+覆盖 Configuration
r.getSavedState(), // 进程死后的恢复状态
results, newIntents, // 待投递的结果和 Intent
...)); // 全部"出生材料"
// 讲解注释⑭：终态（正常启动 → RESUME；启动到后台 → PAUSE）
clientTransaction.setLifecycleStateRequest(
ResumeActivityItem.obtain(isTransitionForward));
// 讲解注释⑮：Binder 下发 → App 进程
mService.getLifecycleManager().scheduleTransaction(clientTransaction);
// ★ 此处第二次 Binder 跳（system_server → App）
}
```

第 ⑦ 步：App 侧接收事务

java App 进程 · binder 线程

```
// ActivityThread.java:1847 → TransactionExecutor.execute
// 讲解注释⑯：先执行 callbacks（LaunchActivityItem）
// → LaunchActivityItem.execute(:93)
// → handleLaunchActivity(:3785)
// → performLaunchActivity(:3546)
// 讲解注释⑰：performLaunchActivity 四步：
// a) getPackageInfo → LoadedApk（类加载器）
// b) mInstrumentation.newActivity(cl, className, intent) — 反射创建
// c) activity.attach(...) — 注入 Context/PhoneWindow/Application
// d) activity.onCreate(icicle) — 你的代码开始执行
// 讲解注释⑱：再执行 lifecycleStateRequest → cycleToPath 补中间态
// CREATED → STARTED → RESUMED（onStart → onResume）
```

第 ⑧ 步：首帧渲染

java App 进程 · 主线程

```
// onResume 后：
```

```
// ViewRootImpl.setView(:1092)
// → Session.addToDisplay → WMS.addWindow（书5 §二）
// → InputChannel 配对注册（书5 §五）
// scheduleTraversals(:2257)
// → Choreographer.postCallback(CALLBACK_TRAVERSAL)
// → 等 VSync → performTraversals(:2286)
// → performMeasure(:2541) → performLayout → performDraw
// → RenderThread 提交 GPU → SurfaceFlinger 合成
// ★ 此处第三次跨进程（App → system_server → SurfaceFlinger）
```

十步总览图

text 十步全景（★=Binder 跳， Socket=socket 跳）

```
App 主线程 system_server binder 线程 zygote
| |
① startActivity ──★→ ② ATMS.startActivity |
③ ActivityStarter |
├ 进程不存在? ── Socket → ④ fork
| | ↓
| ⑤ attachApp ←─★─ App
⑥ realStartActivity |
| |
⑦ receive ←──────★────── scheduleTransaction |
performLaunch |
onCreate/onStart/onResume |
⑧ ViewRootImpl ──★→ WMS.addWindow |
traversals → draw → SF.composite ──★→ 面板 |
| |
⑨ activityIdle ←──★──────────────────────────┘
⑩ TotalTime 记录完毕
```

设计洞察 · 逐段精读：为什么这是全书最重要的图

三次 Binder 跳 + 一次 socket 跳——串起了书2（fork）、书3（Binder 十跳）、书5（窗口/绘制）。一个 Activity 的启动，是整个 Android 系统协作的缩影。面试官问"从 startActivity 到 onCreate 经过了几步"，你要能不看图数出十步并标注每步所在进程。

第二章 ClientTransaction：生命周期事务化

设计洞察 · 学习目标

1. 背出 TransactionExecutor 的执行次序（callbacks 先、终态后、cycleToPath 补中间态）；2. 解释"状态机路径由执行器推导"为什么消灭了乱序问题；3. 说清旋转重建与 configChanges 自处理两条分支分别对应哪个 Item；4. 用"一次不可拆的事务"对比 Android 9 之前的多次可重排调度



图 2 ClientTransaction 模型：callbacks+终态、App 侧 TransactionExecutor 的补态执行与设计动机

2.1 TransactionExecutor 逐段注释（App 侧执行器）

java frameworks/base/core/java/android/app/servertransaction/TransactionExecutor.java（骨架+讲解注释）

```
public void execute(ClientTransaction transaction) {
// 讲解注释①：先跑 callbacks（LaunchActivityItem/NewIntentItem…按序），
// 每个 Item 自己知道该调 ActivityThread 的哪个 handleXxx。
executeCallbacks(transaction);
// 讲解注释②：再执行终态请求。注意 executeLifecycleState 内部会先
// cycleToPath——从当前状态沿状态机"走"到目标状态，中间态一个不跳。
executeLifecycleState(transaction);
...
}
private void executeLifecycleState(ClientTransaction transaction) {
...
// 讲解注释③：pathToTargetState 生成要补的事件序列。
// 例：当前 onCreate 完（CREATED），终态 RESUME → 补 onStart、onResume。
// 这就是"你永远不会收到没有 onStart 的 onResume"的机制保证。
cycleToPath(r, lifecycleItem.getTargetState(), true /* excludeLastState */, transaction);
// 讲解注释④：真正执行终态 Item（ResumeActivityItem→activityResumed），
// Item.postExecute 再回执 AMS（activityIdle 链在此闭环）。
lifecycleItem.execute(mTransactionHandler, token, mPendingActions);
...
}
```

逐段精读：③是整个事务模型的灵魂——**状态机路径由执行器推导**而非调用方拼装。AMS 只说"我要它到 RESUME"，怎么走是协议层保证的。这把 Android 9 之前"调度方拼事件序列"的责任反转成了"执行方补路径"，出错面骤减。

> **设计洞察 · 为什么 Android 9 要改造**
> 旧世界几十个 scheduleXxx 独立 Binder 调用，顺序与原子性全靠约定——配置变更场景常出现"先 resume 后 config"的闪烁。事务化后一次投递一批变更，App 侧 cycleToPath 保证状态机走连续路径（CREATE→RESUME 必经 START），系统侧可整批回放。**把"多次可重排的消息"变成"一次不可拆的事务"，是分布式系统对抗乱序的经典解法。**

App 侧执行器 TransactionExecutor 的规则值得背下：先逐项执行 callbacks（LaunchActivityItem、NewIntentItem…按序），再执行 lifecycleStateRequest（沿状态机补中间态），最后 postExecutionState 回执 AMS。servertransaction/ 目录就是"生命周期指令集"——新增周期能力=新增一个 Item 类，协议不动。

2.2 屏幕旋转：一次"重建事务"的解剖

旋转不是"魔法重启"，而是三条事务的组合拳（ConfigurationChangeItem + ActivityRelaunchItem，均在 servertransaction/）：

- **能自处理**（manifest configChanges=orientation）→ 只发 ActivityConfigurationChangeItem → App 侧 onConfigurationChanged，View 树自己适配。
- **不能自处理** → ActivityRelaunchItem：**保留状态打包（onSaveInstanceState 已提前回调）→ 销毁（DestroyActivityItem）→ 全新 LaunchActivityItem 重走出生仪式 → 恢复 savedInstanceState**。

逐段精读：为什么框架"宁可重建"也不默认增量适配？——任意 View 树都可能持有与配置耦合的资源（尺寸限定符布局、横竖屏两套 layout），增量适配的正确性成本高于重建的性能成本。**Android 在"通用正确"与"个案最优"之间永远选前者，再把优化口子（configChanges）留给开发者**——这是读框架决策的一把万能钥匙。

第三章 Service：最安静的组件

> **设计洞察 · 学习目标**
> 1. 复述 startService 的"欠条-兑现"路径（mPendingServices 等孵化补发）；2. 列出 bind 的三本账（ConnectionRecord/ServiceRecord/AppBindRecord）与 publishService 回传链；3. 背出前后台服务超时线 20s/200s 及熔断三处（次数限流/类型强制/后台启动禁止）；4. 用 onStartCommand 三种返回值解释被杀重启的 payload 差异



图 3 四大组件调度对比：入口/宿主/下发/超时四象限对照

3.1 启动链与"先孵化后补发"

`startService` → `ActiveServices.startServiceLocked`（`ActiveServices.java:675`）→ `bringUpServiceLocked`（`:1179`）：查 ServiceRecord → 进程不在？**先走 ProcessList 孵化**（复用书 2 的 zygote 流水线），把 ServiceRecord 挂到 `mPendingServices`；`attachApplicationLocked` 时补发 `CreateServiceItem` + `StartServiceItem`。这个"欠条-兑现"模式与 Activity 的 pending 事务、Broadcast 的 `mPendingBroadcast`、Provider 的 `mLaunchingProviders` 四处同构——**AMS 对"目标还不存在"的唯一答案就是记账后等孵化回调**。

3.2 bind 的三本账

`bindServiceLocked`（`:2755`）维护：ConnectionRecord（谁绑的）、ServiceRecord（服务本体）、AppBindRecord（进程间绑定桥）。回调 `onBind` 返回的 IBinder 经 `publishServiceLocked` 回传给每个连接方——**onBind 只调一次，onRebind 看意图**。死亡时 `DieConnection` 清账（书3 死亡通知的客户之一）。

3.3 前台服务与超时

前台服务 = 普通服务 + 类型化通知（`foregroundServiceType`）+ 优先级背书（`PROCESS_STATE_BOUND_FOREGROUND_SERVICE`）。超时线：前台 20s / 后台 200s（`SERVICE_TIMEOUT/SERVICE_BACKGROUND_TIMEOUT`），超时走 `serviceTimeout` → AnrHelper。**Android 13 的 FGS 通知延迟语义**（用户离开 10s 后才显示部分类型通知）在本树的 `ActiveServices` 与

SystemUI 通知过滤中实现——读码入口 `fgsNotificationDeferred`。

3.4 startServiceLocked 逐段注释（真实骨架）

java frameworks/base/services/core/.../am/ActiveServices.java:675-1179（骨架+讲解注释）

```
ComponentName startServiceLocked(IApplicationThread caller, Intent service, ...) {
// 讲解注释①：检索 ServiceRecord（首次则创建：组件名→服务账本）。
ServiceLookupResult res = retrieveServiceLocked(...);
...
// 讲解注释②：启动次数限流（后台启动 30s/30 次的熔断，防恶意自启）。
if (forcedStandby || r.startRequested) { ... }
...
// 讲解注释③：进入 bringUpServiceLocked（:1179）——"欠条-兑现"的起点。
ComponentName cmp = startServiceInnerLocked(r.defaultApp, service, ...);
}
private String bringUpServiceLocked(ServiceRecord r, ...) {
// 讲解注释④：进程在 → 直接 realStartServiceLocked：
// scheduleCreateService（onCreate）→ onStartCommand（onStartItem）
// 讲解注释⑤：进程不在 → startProcessLocked 孵化，
// ServiceRecord 挂 mPendingServices，等 attachApplication 兑现。
...
}
```

设计洞察 · 逐段精读：三处熔断

②的次数限流、前台服务类型强制（13 的 foregroundServiceType 必须声明）、后台启动限制（App 处于缓存态时 startService 直接抛 `IllegalStateException`——要求改用 startForegroundService+5s 内 startForeground）。三个限制分别对付：恶意自启、通知失明、后台耗电。**每一个"不能"，背后都有一场攻防史。**

3.5 onStartCommand 返回值：服务的"复活意愿声明"

java 应用侧（讲解注释）

```
public int onStartCommand(Intent intent, int flags, int startId) {
// START_NOT_STICKY：被杀后不重启（除非再有 startService）——一次性任务选它
// START_STICKY：被杀后重启，但重投的 intent 为 null（常驻服务选它）
// START_REDELIVER_INTENT：被杀后重启并重投最后的 intent（关键任务选它）
return START_STICKY;
}
```

兑现机制在 `ActiveServices` 的 `serviceTimeout`/进程死亡清理路径：ServiceRecord 记录 `restartDelay`（指数退避，防止杀-启死循环——`SERVICE_RESTART_DURATION`/`SERVICE_RESET_RUN_DURATION` 常量族）。**AMS 不是简单地"尊重你的声明"，而是给复活加了退避与配额**——又是一处"用户可见语义 × 系统自保"的平衡设计。

第四章 Broadcast：系统的脉搏

设计洞察 · 学习目标

1. 画出双队列流水线并指出动态/有序接收者的分流点；2. 走读 processNextBroadcastLocked 四段（并行出货/有序推进/超时判决/收尾）；3. 解释 goAsync 改变了超时链条的哪一环而没改变超时线本身；4. 说出"单点故障不扩散"在 broadcastTimeoutLocked 里的源码表现

4.1 双队列流水线

`broadcastIntent` 入口把接收者分流：**动态接收者进并行队列**（立即逐个分发，互不等待）；**有序接收者（含 manifest 接收者）进有序队列**（前一个 `finishReceiver` 才轮到下一个）。驱动循环：`scheduleBroadcastsLocked`（`BroadcastQueue.java:468`）→ `processNextBroadcast`（`:1137`）/ `processNextBroadcastLocked`（`:1149`）。

4.2 逐段精读 processNextBroadcastLocked 的四个段落

- **第一段（并行队列出货）**：while 循环把 mParallelBroadcasts 逐条 `deliverToRegisteredReceiverLocked`——每个接收者一个 Binder 调用（`scheduleReceiver`），不等回执。
- **第二段（有序队列推进）**：取队首 BroadcastRecord 的当前 receiver：进程活着→`processCurBroadcastLocked` 下发并**启动超时计时**；进程不在→孵化并挪入 `mPendingBroadcast`（记账等兑现，与 Service 同款）。
- **第三段（超时判决）**：`BROADCAST_TIMEOUT_MSG`（`:203`）到期→`broadcastTimeoutLocked`：最后手段是 ANR（前台 10s/后台 60s）。
- **第四段（收尾）**：所有 receiver 走完→finish receivers、发下一个、必要时触发后台 restriction（动态广播静默丢弃）。

设计洞察 · 面试题眼：为什么有序广播会引发连锁 ANR

一个接收者主线程卡住→finishReceiver 迟到→超时计时到点→整条广播链 ANR。这也是 `goAsync()` 的价值：把耗时挪到自己的工作线程，但**必须在 10s 内调 finish**——它延长的是"处理窗口"不是"超时线"。

4.3 广播下发与超时（真实骨架逐段注释）

java am/BroadcastQueue.java（骨架+讲解注释）

```
// ── processNextBroadcastLocked 有序段落（:1149 起）──
// 讲解注释①：当前 receiver 的进程不在？孵化 + 挂 mPendingBroadcast，
// 与 Service 的 pending 同构（四组件第二处"欠条"）。
// 讲解注释②：在 → processCurBroadcastLocked：
// app.thread.scheduleReceiver(... ReceiverData ...)（一次 Binder 下发）
// + setBroadcastTimeoutLocked(lockStartCommittedElapsed)（挂超时闹钟）
```

```
// —— broadcastTimeoutLocked（超时判决）——
// 讲解注释③：前台队列 10s / 后台 60s（mTimeOutPeriod）；
// 广播期间收到 finishReceiver 会 cancelTimeout——所以 ANR 只属于"真卡住"。
// 讲解注释④：判决 = annotateAppNotResponding（走 AnrHelper 队列），
// 并跳过该 receiver 继续下一位（广播链不断，烂的只是那一环）。
```

逐段精读：④的"跳过继续"常被忽略——**ANR 杀的是进程（用户可见"应用无响应"弹窗），但广播链本身会继续推进**，系统不会因一个僵死接收者停摆。这是"单点故障不扩散"的调度哲学。

第五章 ContentProvider：数据的跨进程门户

设计洞察 · 学习目标

1. 复述 getContentProviderImpl 的"孵化-记账-等待-唤醒"时序与 10s 超时兜底；2. 说清 incProviderCount 引用计数如何抬高宿主进程的缓存优先级；3. 解释"控制面 Binder、数据面共享内存"的性能观（CursorWindow/fd 传递）；4. 走通 publishContentProviders 一揽子发布与 URI 临时授权双闸

`getContentProviderImpl`（`ContentProviderHelper.java:158`）是四组件里唯一**同步阻塞**的入口：发布者进程不在→孵化→**调用方线程持锁等待**（`mLaunchingProviders` 记账），`publishContentProviders` 唤醒。引用计数（`incProviderCount:261`）决定宿主进程的缓存优先级——纯 provider 进程杀得比空 Activity 进程晚的机制根源。

App 侧 `installProvider`（ActivityThread）反射建 Provider 对象并挂 `IContentProvider` 代理。**数据面不走 Binder 大包**：query 返回 Cursor（BulkCursorNative + 窗口共享内存 ashmem/memfd），大文件走 `openAssetFile`（fd 传递，书3 §6）。控制面小、数据面共享内存——这是 Provider 高吞吐的秘密。

5.1 getContentProviderImpl 逐段注释（同步等待的正确姿势）

java am/ContentProviderHelper.java:158-268（骨架+讲解注释）

```
private ContentProviderHolder getContentProviderImpl(IApplicationThread caller, ...) {
synchronized (mService) { // 讲解注释①：全程持 AMS 大锁——Provider 语义要求强一致
// 讲解注释②：按 authority 查 providerMap；无 → 解析 authority 找宿主包。
// 讲解注释③：宿主进程在 → 直接 installProviderLocked，把 holder 交给调用方。
// 讲解注释④：不在 → checkTime 开始计时；startProcessLocked 孵化宿主；
// mLaunchingProviders 记账；调用方 Object.wait() 让出（持锁等待），
// publishContentProviders（宿主 attach 时回调）notifyAll 唤醒。
// 讲解注释⑤：引用计数 incProviderCount（:261）——调用方进程因此
// 在 computeOomAdj 里获得 BOUND_PROVIDER 加权（宿主不容易被杀）。
}
}
```

逐段精读：④的"持锁 wait"与②的"authority 先注册"合起来构成 Provider 的**发布-订阅时序**：宿主 APK 的 provider 在 manifest 声明后由 PMS 索引（`<provider android:authorities=...>`），AMS 按 authority 寻址。超时兜底 CONTENT_PROVIDER_TIMEOUT 10s——挂起不会无限等。

App 侧消费：`installProvider` 反射建对象后，`query` 走 `BulkCursor` 协议（CursorWindow 共享内存分页搬运），大文件 `openFile` 直接 fd 过桥（书3 §6）。**控制面 Binder、数据面共享内存**——记住这句就懂了 Provider 的性能观。

5.2 publishContentProviders 与 URI 授权

宿主进程 attach 时（`ActivityThread.handleBindApplication` 后）调 `publishContentProviders`：把本进程全部 Provider 一次性登记进 providerMap 并唤醒等待者。**发布是一揽子的**——所以一个 App 的多个 provider 要么都可见要么都没有，不存在半发布态。

URI 级授权（`UriGrantsManagerService`，Bootstrap 批启动）提供比 provider 粒度更细的控制：`Intent.FLAG_GRANT_READ_URI_PERMISSION` 临时授权给目标包，过期自动回收（activity destroy 时清账）。**读码提示**：图库分享一张照片走的是"provider 可见性 + URI 临时票据"双闸——不是把整个图库 provider 开放给对方。

第六章 进程优先级：computeOomAdj 记账机器

> **设计洞察 · 学习目标**
>
> 1. 按"短路顺序"背出 computeOomAdj 的十档因子表；2. 区分 PROCESS_STATE 与 oom_score_adj 双刻度的消费方；3. 走通五源 ANR 的触发线与 AnrHelper 统一出口；4. 用 dumpsys/logcat 观测一次 adj 变化并解释 cgroup 与冻结的待遇差异

6.1 computeOomAdj 因子全表（按核算顺序）

#	因子	命中后状态（例）
1	instrumentation/备份/调度器自用	NONSTOP 高位
2	顶层 Activity（TOP_APP）	PROCESS_STATE_TOP
3	可见但非顶层（被透明/PiP 覆盖）	VISIBLE
4	可感知绑定（BIND_VISIBLE 等 flag）	BOUND_FOREGROUND_SERVICE
5	前台服务持有	FOREGROUND_SERVICE
6	正在收重要广播	IMPORTANT_FOREGROUND
7	正在执行后台服务/普通绑定	SERVICE / BOUND
8	上一应用（PREVIOUS_APP）	LAST_ACTIVITY 档
9	缓存 provider 引用/权重提升	BACKUP/HEAVY_WEIGHT
10	都无命中 → 按LRU入缓存池	CACHED_ACTIVITY(_MAX~_MIN)

三句读表心法：**越靠前权重越大**（短路：命中即定档）；**缓存池有档位与容量**（32/32 档 min/max，超额最旧的杀）；**state 与 adj 双刻度**（state 给框架消费，adj 给 lmkd）。



图 4 进程优先级机器：输入因子→process state/oom_adj→cgroup/lmkd，含观测与面试追问链

`OomAdjuster.updateOomAdjLocked`（`OomAdjuster.java:421`）对每个 ProcessRecord 跑 `computeOomAdjLSP`：二十来个因子（顶层/可见/前台服务/正在收广播/缓存 provider/上一应用…）核算出 `PROCESS_STATE_*` 与 `oom_score_adj`，然后三件事：写 `/proc/<pid>/oom_score_adj`、切 cgroup（前台/后台组，决定 CPU 带宽与冻结资格）、把 state 上报给各记账方（JobScheduler 配额、binder 同步限制、缓存冻结书3 § 10.4）。

lmkd（`system/memory/lmkd`）拿着 adj 位图与 PSI 压力事件决定杀谁：从最缓存档开始，先冻结后杀。读码入口 `AnrHelper.appNotResponding`（`:71`）则是 ANR 的统一出口——队列化执行 dumpStackTraces，避免在 binder 线程里做重活（这个细节本身就是"系统线程纪律"的活教材）。

6.2 cgroup 与冻结：adj 之外的两只手

adj 定生死，**cgroup 定待遇，冻结定作息**：

- **CPU 组**：前台组（`/dev/cpuctl/foreground`）与后台组（`background`，`cpu.shares` 约 2:10-20:1024 差距）；Service 客户端绑定会把宿主临时拉进前台组（`setProcessGroup` 调用族）。
- **冻结（12+）**：缓存态进程进 freezer cgroup——CPU 全停、binder 同步调用速败 `BR_FROZEN_REPLY`（书3 § 10.4）、广播直接绕行（缓存接收者广播默认 deferred）。**冻结是"杀"前的缓冲带**：内存压力小就解冻复用（省一次冷启动），压力大就顺势 kill。

lmkd 通信面：AMS 侧 `ProcessList` 经 socket 把 adj 变更推给 lmkd（`LMK_PROCPRIO` 命令），压力事件回流（`onPressureChanged`），击杀经 `LMK_KILL` 进 logcat——`logcat -s lmkd` 看到的每一行都是本节机制的现场直播。

> **源码锚点 · 六册联动锚点**
> ProcessList.java:1899 startProcessLocked →（书2 Zygote 流水线）
OomAdjuster.java:421 updateOomAdjLocked → cgroup/lmkd →（书3 §10.4 冻结）
AnrHelper.java:71 appNotResponding ←（书5 input-pipeline 的 5s 计时器）
ContentProviderHelper.java:158 ←（书6 PMS 的包宇宙支撑解析）

6.3 ANR 完整解剖：五源五链五治（教科书级）

ANR（Application Not Responding）是系统开发者的日常面包——本节把 ANR 从头到尾讲透。

五个触发源

源	超时线	计时起点	代码入口
① 输入	5s	InputDispatcher publishMotion	AnrTracker → notifyANR
② 前台 Service	20s	scheduleCreateService	ActiveServices.serviceTimeout
③ 后台 Service	200s	同上	同上
④ 前台广播	10s	processCurBroadcastLocked	BroadcastQueue.broadcastTimeoutLocked
⑤ 后台广播	60s	同上	同上

统一出口：AnrHelper

java AnrHelper.java:71（骨架+逐行注释）

```
void appNotResponding(ProcessRecord anrProcess, String annotation) {
// 讲解注释①：队列化执行——不在触发线程做重活（dump 全线程栈很重）
// 避免把 binder 线程也卡住
AnrRecord anr = new AnrRecord(anrProcess, annotation);
synchronized (mAnrRecords) {
mAnrRecords.add(anr);
}
startAnrProcessorIfNeeded(); // 后台线程处理
}
// 后台线程的主逻辑：
// 1. Process.dumpStackTraces → /data/anr/traces.txt（全线程栈）
// 2. 判断是否显示弹窗（后台 ANR 不弹）
// 3. 如果 too_many_anrs → 杀进程
// 4. 弹窗："应用无响应" → [等待] [关闭]
```

输入 ANR 的完整链（最高频场景）

text 输入 ANR 五步链

```
① 用户触摸屏幕
→ InputDispatcher.publishMotion → 写入 App 的 InputChannel
→ 计时开始（5 秒倒计时）
② App 主线程正在忙
→ 可能原因：
a）正在执行上一个 onTouchEvent（业务慢）
b）正在等锁（被其他线程持有）
c）正在做 IO（SharedPreferences 写入/数据库查询）
d）正在做 Binder 同步调用（等 system_server 回复）
e）被 GC 暂停（STW 时间长）
③ 5 秒后 App 仍未回执 finishInputEvent
→ AnrTracker 超时 → InputManagerService.notifyANR
④ AMS 收到 notifyANR
→ AnrHelper.appNotResponding（队列化）
→ dump 全线程栈 → /data/anr/traces
⑤ 用户看到弹窗
→ "应用无响应" → [等待] [关闭]
→ 关闭 = 杀进程
```

ANR 排障五步法

bash

```
# 步骤 1：看 traces 找卡住的线程
adb bugreport bugreport.zip # 或
adb pull /data/anr/
# 在 traces.txt 中找 "main" 线程的栈
# 步骤 2：分析主线程栈
# 常见模式：
# - "waiting to lock" → 锁竞争 → 找锁持有者
# - "Binderproxy.transact" → 等 system_server 回复 → 查服务端
# - "SocketInputStream.read" → IO 阻塞 → 看操作的是什么
# - "SharedPreferencesImpl" → 磁盘 IO → 移到子线程
# - "DalvikVM GC" → GC 暂停 → 减少内存分配
# 步骤 3：看其他线程（找锁持有者）
# 在 traces 中搜索持有主线程等待的锁的线程
# 步骤 4：看 logcat 前后日志
adb logcat -d -t 200 | grep -E "ANR|am_anr|Input"
# 步骤 5：修复并复验
# 修复后 → 重新复现场景 → 确认不再 ANR
```

预防 ANR 的五条军规

军规	原因	实践
主线程不做 IO	磁盘 IO 100ms~秒级	SharedPreferences 移到子线程
主线程不做 Binder 同步	服务端可能慢	改 oneway 或异步
减少主线程对象分配	GC 暂停累积	onDraw 里不 new 对象
避免嵌套锁	死锁的主因	统一锁顺序

军规	原因	实践
Service.onCreate 快进快出	20s 硬顶	重活移到子线程

> **设计洞察 · 逐段精读：ANR 弹窗为什么另一个进程弹**
> 将死进程的 UI 不可信——它的主线程已经卡住了。弹窗在 system_server 里发起，经 SystemUI 显示——"**不信任将死进程的 UI"是系统级安全的设计直觉**。

6.4 computeOomAdj 完整教程（含所有因子实战判断）

这是 AMS 最复杂的单体函数（~2000 行）——本节用"输入→决策→输出"框架讲透。

因子判断的短路顺序（先到先得）

java OomAdjuster.java:421 updateOomAdjLocked（骨架+注释）

```
// 讲解注释①：以下因子按优先级从高到低排列——
// 命中任何一个就"短路"（后续因子不再检查）
// 因子 1：instrumentation/备份（最高优先）
if (isInstrumentation) → PROCESS_STATE_NONSTOP_ENFORCED
// 因子 2：顶层 Activity（用户正在看）
if (r == topApp) → PROCESS_STATE_TOP // adj = 0
// 因子 3：可见但非顶层（被透明 Activity 覆盖 / PiP）
if (r.visible) → PROCESS_STATE_VISIBLE // adj ≈ 100
// 因子 4：可感知绑定（BIND_VISIBLE 等 flag 的 Service 绑定者）
if (hasVisibleBinding) → PROCESS_STATE_BOUND_FOREGROUND_SERVICE
// 因子 5：前台服务持有者
if (hasForegroundService) → PROCESS_STATE_FOREGROUND_SERVICE
// 因子 6：正在收重要广播
if (isReceivingBroadcast) → PROCESS_STATE_IMPORTANT_FOREGROUND
// 因子 7：正在执行后台 Service / 普通绑定
if (hasActiveServices || hasBoundServices) → PROCESS_STATE_SERVICE
// 因子 8：上一应用（刚切走的）
if (r == previousProcess) → PROCESS_STATE_LAST_ACTIVITY
// 因子 9：缓存 provider 引用 / 权重提升
if (hasCachedProvider || hasWeightBalancing) → 提升
// 因子 10：都无命中 → 进入缓存池
→ PROCESS_STATE_CACHED_ACTIVITY // 按 LRU 排序
```

双刻度输出

输出	消费方	范围
PROCESS_STATE_*	JobScheduler/binder 冻结/配额	TOP(0)~CACHED_ACTIVITY_MIN(~16)
oom_score_adj	/proc/pid/oom_score_adj → lmkd	0(前台)~1000(最先杀)

实战：观察进程优先级变化

bash

```
# 实时观察你的 App 的 adj 变化
adb shell am monitor -p com.example.app &
# 切前台/切后台/播放音乐/收广播 → 观察 state 变化
# 查看 lmkd 的杀进程日志
adb logcat -s lmkd
# 输出：Kill 'com.example.app' (12345) to free xxx kB
# adj: 980 → 这就是当时的 oom_score_adj
# dumpsys 视角
adb shell dumpsys activity processes | grep -A5 "com.example.app"
# 输出：oom adj=980 state=cached/empty
```

面试常考场景

场景	process state	adj	谁触发
正在用 App	TOP	0	因子2
按 Home 切到桌面	LAST_ACTIVITY	~100	因子8
后台播放音乐	BOUND_FOREGROUND	~100	因子4
收到推送正在处理	IMPORTANT_FOREGROUND	~125	因子6
完全后台 1 小时	CACHED_ACTIVITY	~900+	因子10
缓存池满+低内存	--	--	lmkd 杀掉

设计洞察 · 逐段精读：保活为什么难

从因子表可以看出：**每个能提升优先级的因子都需要系统资源背书**——顶层需要界面、前台服务需要通知、可感知绑定需要可见的绑定方。App 自己"想保活"没有任何因子可以命中——优先级由**服务端核算**（不是 App 自报）。这就是 Android 后台治理的机制根基。

第七章 深水区：ART 内存执行系统与 lmkd

设计洞察 · 学习目标

1. 说清 ART CC 收集器"读屏障+并发搬移"如何把 STW 压到毫秒级；2. 解释 MTERP 汇编解释器、JIT、AOT 三级火箭的分工与 profile 反哺环；3. 复述 lmkd 的"AMS 出价、内核报时、lmkd 落槌"三角；4. 指出 CachedAppOptimizer 冻结与 cgroups.json/task_profiles.json 单一事实源的关系

7.1 ART GC：并发复制收集器（CC）

主收集器是**并发复制**（`art/runtime/gc/collector/concurrent_copying.cc`，堆按 RegionSpace 的 region 组织）：标记与对象搬移大部分与应用线程并发，STW 只剩根初始标记与最终修正两个小阶段。支撑三件套：**读屏障**（引用读取顺带看 forwarding 指针、按需搬移对象——搬移不停止世界）、**卡表/记忆集**（跨代引用脏页索引，免全堆扫）、**LOS 大对象空间**（独立回收防碎片

拷贝）。观测投影即 `dumpsys meminfo` 的各 space 行。

> **设计洞察 · 面试一句话**
> Dalvik 是标记-清除 STW；ART CC 把停顿压到毫秒级，代价是每条引用读都带屏障——**用 CPU 换延迟**，与 Binder 用一次拷贝换安全是同一种设计味道。

7.2 MTERP：为什么解释器用汇编写

`art/runtime/interpreter/mterp/`（arm64ng）：每条字节码一个汇编 handler，取指-分发-执行全内联、虚拟寄存器钉物理寄存器——不走 C 调用栈，吞吐近翻倍。它不是 JIT（仍逐条解释），是把"解释的地板"垫高；JIT（`art/runtime/jit/`）在此之上按轮廓计数挑热点进 code cache，profile（书1 §8.1 的反哺环）供 AOT 精编。**解释器-快冷、JIT-快糙、AOT-慢精**三级火箭各司其职。

7.3 lmkd：压力驱动的击杀守护

`system/memory/lmkd/lmkd.cpp:3735` main：订阅内核 **PSI** 压力事件（libpsi）+ 接收 AMS 的 adj 变更（LMK_PROCPRIO 命令族）；压力升级时按 adj 位图从最缓存档选受害者，配合 per-uid 约束与 swap 水位。一句话：**AMS 出价（adj）、内核报时（PSI）、lmkd 落槌**。`logcat -s lmkd` 的 Kill 行即成交记录。

7.4 冻结实现与 cgroup 拓扑

CachedAppOptimizer（am/CachedAppOptimizer.java）：缓存进程整体迁入 freezer cgroup（v2），调度组件前 AMS 解冻（宽限窗口）；冻结态的 binder 行为即书3 §10.4。**cgroup 单一事实源**：`system/core/libprocessgroup/profiles/cgroups.json`（层级挂载）+ `task_profiles.json`（命名档：ProcessCapacityHigh 等——书2 zygote rc 里的 `task_profiles` 行引用的就是它）。**zram**：`rootdir/init.rc:1074-1076` 暴露 idle/writeback，system_server 管理回写——压缩换内存的标配。

> **源码锚点 · 本章锚点**
> art/runtime/gc/collector/concurrent_copying.cc CC 主体 / interpreter/mterp/arm64ng/ 汇编解释器 / jit/
system/memory/lmkd/lmkd.cpp:3735 main + libpsi
am/CachedAppOptimizer.java 冻结状态机
libprocessgroup/profiles/{cgroups,task_profiles}.json cgroup 双源

第八章 线程发动机：Handler/Looper/MessageQueue 全解

> **设计洞察 · 学习目标**
> 1. 背出三件套分工与 dispatchMessage 的两级回退；2. 说清"消息按执行时刻入队"与 nativePollOnce/nativeWake 的 epoll 底座；3. 举例 system_server 主循环、同步屏障、Choreographer 是同一台发动机的不同挂挡；4. 解释 binder 线程与 Handler 线程的桥接（ApplicationThread 落点）

本丛书在书2（`Looper.prepareMainLooper`）与书5（同步屏障）两处直接使用 Handler 体系却未拆解它。它既是 App 开发者的老朋友，也是 system_server 全部服务的心脏——值得一次讲透。

8.1 三件套的分工与源码骨架

java frameworks/base/core/java/android/os（骨架+讲解注释）

```
// Handler：发件与收件一体
public boolean sendMessageAtTime(Message msg, long uptimeMillis) { // :719
// 讲解注释①：消息按"内核单调时钟的执行时刻"入队（不是 delay!），
// MessageQueue 按 when 排序——定时消息与普通消息同一条队。
...
}
public void dispatchMessage(Message msg) { // :97
// 讲解注释②：回调优先级：msg.callback(Runnable) > mCallback > handleMessage()。
// post(Runnable) 与 sendMessage 的差异就消失在这两级回退里。
}
// Looper.loop()（:260）：for(;;) { queue.next(); msg.target.dispatchMessage(msg); }
// —— 主线程"死循环不卡 UI"的真相：循环里的 next() 会睡眠（下节）。
```

8.2 关键一层：Java 队列的底部是 native epoll

java MessageQueue.java（节选+讲解注释）

```
private native void nativePollOnce(long ptr, int timeoutMillis); // :72
private native static void nativeWake(long ptr); // :73
...
nativePollOnce(ptr, nextPollTimeoutMillis); // :335 next() 内：无消息→睡到下个 when
...
nativeWake(mPtr); // :443 enqueueMessage 唤醒（若有更早者/睡眠中）
```

往下再一层（`core/jni/android_os_MessageQueue.cpp:41` NativeMessageQueue → `system/core/libutils/Looper.cpp:181` pollOnce → `:245 epoll_wait`）：MessageQueue 持有 native Looper，epoll 监听三样——wake 管道（nativeWake 写它）、空闲处理器 fd、以及**别的线程塞进来的自定义 fd**（这正是 InputChannel 能被 ViewRootImpl 收到事件的底座：书5 §5.2 的输入回调就是"消息队列顺带 epoll 了 socket"）。所以准确地说：**Handler 是"用 epoll 武装的优先级队列"**；书5 同步屏障插队、书2 system_server 主循环、Choreographer 的 vsync 消息（异步消息）都是这同一台发动机的不同挂挡。

8.3 系统侧的三种用法对照

用法	现场	要点
system_server 主线程	书2 :860 prepareMainLooper	服务构造在主线程、binder 事务转主线程执行
HandlerThread	AMS/ATMS 各线程域	后台线程自带 Looper（binder 线程≠Handler 线程，桥靠 post）
binder→handler 中转	ApplicationThread:1847	binder 线程收事务→主线程 handleMessage（ClientTransaction 的 App 侧落点）

源码锚点 · 本节锚点

Handler.java:97/719 / Looper.java:260 / MessageQueue.java:72/73/335/443
android_os_MessageQueue.cpp:41/107 / libutils/Looper.cpp:181/245
衔接：书5 §3.1 同步屏障 = 本节队列的"异步消息优先"开关

第九章 设计决策八问

9.5 三个高频补全：PendingIntent、onActivityResult、动态注册桥

PendingIntent（`PendingIntent.java:432` getActivity 族）：把"意图+创建者身份+权限"打包成可跨进程转交的凭据——AMS 侧 `getIntentSender` 落账为 **PendingIntentRecord**（`am/PendingIntentRecord.java:54`，`IIntentSender.Stub`）。通知/闹钟/Widget 持有它，触发时由持有者进程调用 send，**以创建者（通常是你 App）的身份与权限执行**——"代扣授权"的系统级实现。安全要点：IntentSender 可取消（cancel）、可变性（FLAG_IMMUTABLE，13 默认强制）、越权面（持凭据者即得你的身份）。

onActivityResult 回传链：startActivityForResult 的 requestCode 随 LaunchActivityItem 到达目标（ActivityRecord 记账）；目标 `setResult+finish` → AMS 生成 **ActivityResultItem**（servertransaction/ 目录，与 LaunchActivityItem 同族）在下次 resume 发起方时随事务下发 → `ActivityThread.handleSendResult` → onActivityResult。所以"结果回来的时机=你重新可见的时机"——背后的执行器还是书4 §2 的同一台事务机器。

动态注册桥（ReceiverDispatcher）：`registerReceiver` 时 LoadedApk（`core/java/android/app/LoadedApk.java:107`）为每个 Context+Receiver 组合建 **ReceiverDispatcher**（内部类）：持 IIntentReceiver binder stub + 一个 Handler（默认主线程）。BroadcastQueue 分发到它 → binder → dispatcher 的 Handler post → 主线程 onReceive。这解释了两个高频事实：**动态接收者永远在主线程回调**（onReceive 里别干重活）与**注册与分发靠 dispatcher 记账防泄漏**（unregister 才拆 binder）。Service 侧同构的 ServiceDispatcher 管绑定回调。

- **为什么四大组件都要"记账-孵化-补发"？** 目标进程可能不存在；事务性调度把"不存在"变成可兑现的欠条，而不是失败。
- **为什么 Activity 调度拆到 ATMS/wm？** 2019 分家：窗口/焦点语义与进程调度解耦（书5 双视图架构），AMS 只管进程，ATMS 管画面。
- **为什么事务化生命周期？** 抗乱序、抗闪烁、批量化——一次 Binder 说清"发生什么+终到哪"。
- **为什么 Provider 同步等待而其他组件异步？** 调用方语义上就要数据（query 返回值），无法欠条化；代价是 10s 超时兜底。
- **为什么广播分双队列？** 动态接收者在乎吞吐（并行），有序接收者在乎次序（串行）——两种业务诉求两种队列。
- **为什么 oom_adj 由服务端核算？** 若 App 自报，保活将不可治理；核算所需的可见性/绑定关系只有 AMS 视角完整。
- **为什么 ANR 统一走 AnrHelper 队列？** ANR 处理本身重（dump 全线程栈），在触发线程同步做会放大卡顿——系统也遵守"主线程不做重活"。
- **为什么 onStartCommand 要三种返回值？** 复活意愿是业务语义，只有开发者知道；框架负责的是给它加上退避与配额的护栏。
- **为什么旋转默认重建？** 通用正确性优先于个案性能，优化口子交给 configChanges 声明。
- **为什么缓存进程冻结而非直接杀？** 冷启动是最贵的用户可感知操作；冻结是"可逆的杀"，内存压力决定是否兑现。
- **为什么 bindService 回调要走 ServiceConnection 而非返回值？** 绑定是异步语义（进程可能要孵化），回调给了框架调度自由。
- **为什么前台服务要通知？** 用户必须能看见"谁在耗电运行"；通知是优先级背书的对价——13 的延迟显示只是弱化打扰，不是取消透明度。

第十章 实战：读一次真实启动的 trace

把 `am start -W` 与 systrace 拼起来读（示例输出，字段名真实）：

text am start -W 输出解读

```
Status: ok
LaunchState: COLD ← 冷启动（另有 WARM/HOT）
Activity: com.example/.MainActivity
TotalTime: 1420 ← 应用可见总耗时（首帧口径）
WaitTime: 1533 ← 调用方视角（am 进程等待总时长）
Complete: true
```

对应 systrace 轨道逐段认领：`am_proc_start`（孵化，§1 图④）→ `ActivityManager: activityStart`（⑥⑦）→ `inflate/Choreographer#doFrame *first*`（⑧⑨，衔接第五册）→ `am_proc_died`（若被杀）。**把 TotalTime 按轨道切成"孵化/事务/首帧"三段**，你就拥有了对任何启动缓慢的定位能力——这也是性能优化团队的第一课。

一个练习闭环：在测试机反复 `am start -W` 同一 App，记录五组 TotalTime；然后 `pm clear` 再测（首启动 vs 热数据）；最后对照本册十步图解释每组差异来自哪几步。

第十一章 Service 完整生命周期：从 startService 到销毁的全本账

设计洞察 · 学习目标

1. 画出 startService 从调用方到 onCreate/onStartCommand 的完整链并标注每步所在进程；2. 说清 bumpServiceExecutingLocked 的 executingStart 与 20s/200s 超时闹钟的记账关系；3. 复述 startForegroundService 的 30s 追责窗口与再 10s 后 ANR 的两段式追责链；4. 走通 stopService/unbindService 汇入 bringDownServiceLocked 的销毁合流，以及 onStartCommand 三种返回值背后的指数退避复活机器

第三章给出了 Service 的骨架（`startServiceLocked` → `bringUpServiceLocked` 的"欠条-兑现"），本章把这条链走到底：真实源码逐段注释、两台超时闹钟的上弦点、销毁路径的三路合流、以及"复活意愿声明"背后的退避机器。读完本章，`dumpsys activity services` 的每一行输出你都该能对上源码字段。

11.1 全链鸟瞰：两本总账

Service 的调度状态摊在两本账上：

- **ServiceRecord（服务本体账）**：`pendingStarts`——待投递的 onStartCommand 欠条队列；`bindings`——按 Intent 分组的 IntentBindRecord 绑定账；`fgRequired/fgWaiting`——前台服务义务追踪位；`executingStart`——正在执行回调的开始时刻（超时判决的锚点）。
- **ProcessServiceRecord（进程侧账，字段名 psr）**：`executingServices`——**20s/200s 超时闹钟的扫描对象**；`executeNesting`——嵌套执行计数，归零时才撤闹钟。

调用链总览：`Context.startService` → Binder → `ActiveServices.startServiceLocked`（:675）→ `startServiceInnerLocked`（:866，fgRequired 与延迟启动记账）→（本树有两个同名重载，ServiceMap 版最终汇入）→ `bringUpServiceLocked`（:4148）→ 进程活着：`realStartServiceLocked`（:4332）；进程不在：`startProcessLocked` 孵化 + 挂 `mPendingServices`，等第一章 1.6 的"总兑现"时刻补发。

11.2 startServiceInnerLocked：fgRequired 背书与延迟启动

java frameworks/base/services/core/java/com/android/server/am/ActiveServices.java:866-968（节选+讲解注释）

```
// 讲解注释①：ServiceRecord 记账：置"要求前台"位，并给本次 start 生成
// StartItem 挂入 pendingStarts——它是未来 onStartCommand(intent) 的欠条；
// startId 单调递增，stopSelfResult(startId) 靠它防止"旧欠条误杀新服务"。
r.fgRequired = fgRequired;
r.pendingStarts.add(new ServiceRecord.StartItem(r, false, r.makeNextStartId(),
```

```
service, neededGrants, callingUid));
if (fgRequired) {
// 讲解注释②：startForegroundService 路径：立刻在 procstats 记前台态、
// 在 AppOps 记 OP_START_FOREGROUND 操作——系统账本上"你已经在前台
// 跑了"，11.6 的追责链从这里就开始铺账。
synchronized (mAm.mProcessStats.mLock) {
final ServiceState stracker = r.getTracker();
if (stracker != null) {
stracker.setForeground(true, mAm.mProcessStats.getMemFactorLocked(),
SystemClock.uptimeMillis()); // 用当前时间，而非 lastActivity
}
}
mAm.mAppOpsService.startOperation(..., AppOpsManager.OP_START_FOREGROUND,
r.appInfo.uid, r.packageName, ...);
}
final ServiceMap smap = getServiceMapLocked(r.userId);
boolean addToStarting = false;
if (!callerFg && !fgRequired && r.app == null
&& mAm.mUserController.hasStartedUserState(r.userId)) {
ProcessRecord proc = mAm.getProcessRecordLocked(r.processName, r.appInfo.uid);
if (proc == null || proc.mState.getCurProcState() > PROCESS_STATE_RECEIVER) {
// 讲解注释③：后台启动第一道闸——调用方不在前台、目标进程不存在
// 或太靠后（比 PROCESS_STATE_RECEIVER 还低）→ 进入延迟启动评估。
if (r.delayed) {
// 讲解注释④：已在延迟队列：这次 start 不重复排队，直接返回。
return r.name;
}
if (smap.mStartingBackground.size() >= mMaxStartingBackground) {
// 讲解注释⑤：正在后台启动的服务数达到上限 → 挂入
// mDelayedStartList，等某个在途启动完成、bringUpServiceLocked
// 被再次调用时兑现（③③的 delayed 位在 11.3 被摘除）。
Slog.i(TAG_SERVICE, "Delaying start of: " + r);
smap.mDelayedStartList.add(r);
r.delayed = true;
return r.name;
}
addToStarting = true;
}
}
...
// 讲解注释⑥：转入 ServiceMap 版重载，最终调用 bringUpServiceLocked。
// 若宿主包处于冻结态（packageFrozen），请求记入 mPendingBringups，
// 解冻后重放——"包冻结期间不拉起组件"的语义由此实现。
ComponentName cmp = startServiceInnerLocked(smap, service, r, callerFg, addToStarting,
callingUid, wasStartRequested);
return cmp;
```

逐段精读：②与 11.6 的追责链是同一个故事的前后两半——**AppOps 的 OP_START_FOREGROUND 记账在 start 那一刻就发生了**，而不是等 startForeground() 才记；所以"系统认为你在前台跑"与"你真的调了 startForeground"之间有一段系统说了算的窗口。③⑤的延迟启动是一台**流量整形器**：系统开机或收大量广播引发的连锁 startService（如各 App 的连接服务）会在 `mMaxStartingBackground` 的闸口排队，避免 zygote 被瞬时 fork 打爆——这与第一章 1.5 的任务栈裁决互为镜像：**Activity 的裁决在"去哪个法庭"，Service 的裁决在"现在能不能上路"**。

11.3 bringUpServiceLocked：进程在与不在的两条岔路

java frameworks/base/services/core/java/com/android/server/am/ActiveServices.java:4148-4301（节选+讲解注释）

```
private String bringUpServiceLocked(ServiceRecord r, int intentFlags, boolean execInFg,
boolean whileRestarting, boolean permissionsReviewRequired, boolean packageFrozen,
boolean enqueueOomAdj) throws TransactionTooLargeException {
if (r.app != null && r.app.getThread() != null) {
// 讲解注释①：宿主进程已活着 → 不再走创建，把 pendingStarts 里
// 积压的欠条打包下发（onStartCommand 批量补发，一次 Binder 一批）。
sendServiceArgsLocked(r, execInFg, false);
return null;
}
if (!whileRestarting && mRestartingServices.contains(r)) {
// 讲解注释②：正在等重启调度的服务不重复拉起——复活机器接管中
// （11.8），此刻再 start 只是延长闹钟，不是立刻起。
return null;
}
if (mRestartingServices.remove(r)) {
clearRestartingIfNeededLocked(r);
}
if (r.delayed) {
// 讲解注释③：从延迟队列摘除——时机成熟，现在就启动。
getServiceMapLocked(r.userId).mDelayedStartList.remove(r);
r.delayed = false;
}
if (!mAm.mUserController.hasStartedUserState(r.userId)) {
// 讲解注释④：用户已停止 → 组件不许起，直接走销毁记账。
String msg = "Unable to launch app " + r.appInfo.packageName + " ... user "
+ r.userId + " is stopped";
Slog.w(TAG, msg);
bringDownServiceLocked(r, enqueueOomAdj);
return msg;
}
// 讲解注释⑤：服务即将被用 → 把宿主包从 stopped 态捞回来。stopped 包
// 不响应隐式启动——PMS 的 stopped-state 账本在此被 AMS 反向修改。
AppGlobals.getPackageManager().setPackageStoppedState(r.packageName, false, r.userId);
...
if (!isolated) {
app = mAm.getProcessRecordLocked(procName, r.appInfo.uid);
if (app != null) {
if (thread != null) {
try {
app.addPackage(r.appInfo.packageName, r.appInfo.longVersionCode,
mAm.mProcessStats);
// 讲解注释⑥：进程在 → 直接兑现 onCreate/onStartCommand。
realStartServiceLocked(r, app, thread, pid, uidRecord, execInFg,
enqueueOomAdj);
return null;
} catch (RemoteException e) {
Slog.w(TAG, "Exception when starting service ...", e);
}
// 讲解注释⑦：DeadObjectException → 不抛给调用方，
// 落到下面的孵化分支重试——对调用方屏蔽一次进程死亡。
}
```

```
}
}
// 讲解注释⑧：进程不在 → 孵化（书2 的 zygote 流水线），HostingRecord
// 记 HOSTING_TYPE_SERVICE；随后 ServiceRecord 挂 mPendingServices，
// 等 attachApplicationLocked 的"总兑现"时刻补发（第一章 1.6）。
app = mAm.startProcessLocked(procName, r.appInfo, true, intentFlags,
hostingRecord, ZYGOTE_POLICY_FLAG_EMPTY, false, isolated);
...
if (r.fgRequired) {
// 讲解注释⑨：startForegroundService 孵化路径 → 给 uid 上临时豁免
// （tempAllowlist），时长正好是 30s 追责窗口——保证 App 有能力
// 在窗口内调起 startForeground 而不被后台限制卡死。
mAm.tempAllowlistUidLocked(r.appInfo.uid,
mAm.mConstants.mServiceStartForegroundTimeoutMs, REASON_SERVICE_LAUNCH,
"fg-service-launch", ...);
}
if (!mPendingServices.contains(r)) {
mPendingServices.add(r);
}
...
```

逐段精读：⑦是极易被忽略的**容错边界**——realStartServiceLocked 里 scheduleCreateService 若撞上 DeadObjectException，本层 catch 后继续走孵化，调用方完全无感。⑨解释了一个真实行为：**startForegroundService 的 App 在 30s 窗口内天然持有临时豁免**（能拉起前台服务、绕过部分后台限制）——这是系统"先给权限、后追责"的设计：不给你 30 秒的执行能力，追责就成了冤案。

11.4 realStartServiceLocked：兑现 onCreate 的完整现场

java frameworks/base/services/core/java/com/android/server/am/ActiveServices.java:4332-4437（节选+讲解注释）

```
private void realStartServiceLocked(ServiceRecord r, ProcessRecord app,
IApplicationThread thread, int pid, UidRecord uidRecord, boolean execInFg,
boolean enqueueOomAdj) throws RemoteException {
r.setProcess(app, thread, pid, uidRecord);
// 讲解注释①：重启计时归零：restartTime/lastActivity 都钉在现在。
r.restartTime = r.lastActivity = SystemClock.uptimeMillis();
final ProcessServiceRecord psr = app.mServices;
final boolean newService = psr.startService(r);
// 讲解注释②：压入 executingServices 执行栈——这一下同时上弦了
// 20s/200s 闹钟（executingStart 在 bumpServiceExecutingLocked:3644
// 记下，serviceTimeout:5725 拿它比对超时线）。
bumpServiceExecutingLocked(r, execInFg, "create", null /* oomAdjReason */);
mAm.updateLruProcessLocked(app, false, null);
updateServiceForegroundLocked(psr, /* oomAdj= */ false);
// 讲解注释③：**先重算优先级、再下发事务**——保证 App 收到 onCreate
// 时，它进程的 PROCESS_STATE 已是 SERVICE 档（JobScheduler、冻结等
// 消费方不会看到"进程在跑服务但账面还是缓存"的错位瞬间）。
mAm.enqueueOomAdjTargetLocked(app);
mAm.updateOomAdjLocked(app, OomAdjuster.OOM_ADJ_REASON_START_SERVICE);
boolean created = false;
try {
...
// 讲解注释④：真正的 Binder 下发：scheduleCreateService 携带
// CreateServiceData（serviceInfo/token/compat 信息）→ App 侧
```

```
// handleCreateService（ActivityThread.java:4477）反射建对象并调
// onCreate。r.postNotification() 处理前台服务通知。
thread.scheduleCreateService(r, r.serviceInfo,
mAm.compatibilityInfoForPackage(r.serviceInfo.applicationInfo),
app.mState.getReportedProcState());
r.postNotification();
created = true;
} catch (DeadObjectException e) {
Slog.w(TAG, "Application dead when creating service " + r);
mAm.appDiedLocked(app, "Died when creating service");
throw e; // 讲解注释⑤：进程死了 → 上抛，11.3⑦ 兜住走孵化重试。
} finally {
if (!created) {
// 讲解注释⑥：失败清理：执行栈出栈（撤闹钟的计数）、
// ServiceRecord 与进程解绑，并调度重启（11.8）。
final boolean inDestroying = mDestroyingServices.contains(r);
serviceDoneExecutingLocked(r, inDestroying, inDestroying, false);
if (newService) {
psr.stopService(r);
r.setProcess(null, null, 0, null);
}
if (!inDestroying) {
scheduleServiceRestartLocked(r, false);
}
}
}
// 讲解注释⑦：绑定请求排队：逐个 scheduleBindService（onBind）。
requestServiceBindingsLocked(r, execInFg);
updateServiceClientActivitiesLocked(psr, null, true);
// 讲解注释⑧：若服务处于 started 态但没有待投递的参数（例如重启
// 场景），伪造一条 intent 为 null 的 StartItem——保证 onStartCommand
// 一定会被调（START_STICKY 重启后 intent==null 的根源就在这）。
if (r.startRequested && r.callStart && r.pendingStarts.size() == 0) {
r.pendingStarts.add(new ServiceRecord.StartItem(r, false, r.makeNextStartId(),
null, null, 0));
}
// 讲解注释⑨：批量下发全部欠条 → onStartCommand（一次或多次）。
sendServiceArgsLocked(r, execInFg, true);
if (r.delayedStop) {
// 讲解注释⑩：启动途中已被要求停止（stopService 早于 onCreate
// 到达的经典竞态）→ 延迟停止位置真，现在补刀 stopServiceLocked。
r.delayedStop = false;
if (r.startRequested) {
stopServiceLocked(r, enqueueOomAdj);
}
}
}
```

设计洞察 · 逐段精读：⑧⑨两个动作合起来回答了一道经典面试题

为什么 START_STICKY 的服务被杀重启后，onStartCommand 收到的 intent 是 null？因为重启路径走 realStartServiceLocked 时 pendingStarts 已空，框架**伪造了一条无参欠条**（⑧），只为把 onStartCommand 的调用序补全；START_REDELIVER_INTENT 的区别则在于 sendServiceArgsLocked 会把 deliveredStarts 里未完成的 StartItem 连同原 intent 一起重投（START_FLAG_REDELIVERY 标志，ActivityThread.handleServiceArgs:4673 处理）。**"被杀重启"不是神秘复活，而是同一套欠条账本的再演。**

11.5 App 侧到站：handleCreateService 与回执

App 侧 `handleCreateService`（ActivityThread.java:4477）四步：`makeApplicationInner` 确保 Application 存在 → `getAppFactory().instantiateService` 反射建对象 → `service.attach(context, this, name, token, app, AMS)` 注入 Context 与 token → `service.onCreate()`。随后立刻 `serviceDoneExecuting(data.token, SERVICE_DONE_EXECUTING_ANON, 0, 0)` 回执 system_server——**onCreate 是"同步快进快出"的契约**：这个回执让 executingServices 出栈，20s 闹钟随之撤下（第六章 6.3 军规"Service.onCreate 快进快出"的机制根源）。onStartCommand 的处理在 `handleServiceArgs`（:4673），每条 StartItem 处理完同样回执 `SERVICE_DONE_EXECUTING_START`。

11.6 两台超时闹钟：20s/200s 与 30s+10s

第一台：回调执行闹钟。 压栈时 `bumpServiceExecutingLocked` 记下 `executingStart`，出栈归零时撤闹钟；`scheduleServiceTimeoutLocked`（:5827）按 `shouldExecServicesFg()` 上弦 20s（`SERVICE_TIMEOUT:209`）或 200s（`SERVICE_BACKGROUND_TIMEOUT:212`，= 20s × 10）。

java frameworks/base/services/core/java/com/android/server/am/ActiveServices.java:5712-5761（节选+讲解注释）

```
void serviceTimeout(ProcessRecord proc) {
String anrMessage = null;
synchronized (mAm) {
if (proc.isDebugging()) {
return; // 讲解注释①：调试中不判超时——断点不是罪。
}
final ProcessServiceRecord psr = proc.mServices;
if (psr.numberOfExecutingServices() == 0 || proc.getThread() == null) {
return; // 讲解注释②：执行栈已空 → 闹钟是迟到的，无罪释放。
}
final long now = SystemClock.uptimeMillis();
final long maxTime = now -
(psr.shouldExecServicesFg() ? SERVICE_TIMEOUT : SERVICE_BACKGROUND_TIMEOUT);
// 讲解注释③：从栈顶往下找第一个 executingStart 早于
// maxTime 的记录——它就是超时嫌疑犯（最老的未归栈回调）。
ServiceRecord timeout = null;
long nextTime = 0;
for (int i = psr.numberOfExecutingServices() - 1; i >= 0; i--) {
ServiceRecord sr = psr.getExecutingServiceAt(i);
if (sr.executingStart < maxTime) {
timeout = sr;
break;
```

```
}
if (sr.executingStart > nextTime) {
nextTime = sr.executingStart;
}
}
if (timeout != null && mAm.mProcessList.isInLruListLOSP(proc)) {
Slog.w(TAG, "Timeout executing service: " + timeout);
...
anrMessage = "executing service " + timeout.shortInstanceName;
} else {
// 讲解注释④：要么没有嫌疑犯、要么进程已不在 LRU 列表
//（即将被清）→ 只把闹钟顺延到下一个回调的到期时刻。
Message msg = mAm.mHandler.obtainMessage(
ActivityManagerService.SERVICE_TIMEOUT_MSG);
msg.obj = proc;
mAm.mHandler.sendMessageAtTime(msg, psr.shouldExecServicesFg()
? (nextTime+SERVICE_TIMEOUT) : (nextTime + SERVICE_BACKGROUND_TIMEOUT));
}
}
if (anrMessage != null) {
// 讲解注释⑤：判决落地——走 AnrHelper 统一出口（第六章 6.3）。
mAm.mAnrHelper.appNotResponding(proc, anrMessage);
}
}
```

第二台：startForeground 追责闹钟。 `startForegroundService` 的义务（fgRequired）在**首次投递 onStartCommand 时**上弦（`sendServiceArgsLocked:4478` → `scheduleServiceForegroundTransitionTimeoutLocked:5838`，`r.fgWaiting = true`，时长 `mServiceStartForegroundTimeoutMs` 默认 30s，见 `ActivityManagerConstants:231`）。到点后 `serviceForegroundTimeout`（`:5763`）做两件事：**先** `stopServiceLocked` 把没尽义务的服务停掉，**再**发一条延迟 `mServiceStartForegroundAnrDelayMs`（默认 10s，`ActivityManagerConstants:233`）的 ANR 消息 → `serviceForegroundTimeoutANR:5797`，ANR 原文就是你在 logcat 里见过无数次的 *"Context.startForegroundService() did not then call Service.startForeground()"*。另有一条岔路：若服务在 fgRequired 期间被 bringDown（例如调用方 stopService），`bringDownServiceLocked:4610` 直接发 `SERVICE_FOREGROUND_CRASH_MSG` → `serviceForegroundCrash:5818` 抛 `ForegroundServiceDidNotStartInTimeException`。

闹钟	上弦点	时长	到点动作
回调执行（onCreate/onStartCommand/onBind 等在栈未归）	bumpServiceExecutingLocked:3644	前台 20s/后台 200s	serviceTimeout:5712 → AnrHelper ANR
startForeground 义务	sendServiceArgsLocked:4478	30s（可配）	先 stopService，再 10s 后 ANR
义务期内被销毁	bringDownServiceLocked:4610	—	直接抛 ForegroundServiceDidNotStartInTimeException

设计洞察 · 设计洞察：为什么追责要拆成 30s+10s 两段

stopService 与 ANR 分离，是"**止损能止损、追责归追责**"：先停服务让系统资源立刻回收（用户不再为失职服务买单），再给 10s 缓冲收集证据走 ANR 流程——ANR 处理本身是重活（dump 全线程栈，第六章 6.3），叠在止损动作上会把两个职责搅在一起。这与 AnrHelper 队列化的思路同源：**惩罚动作自身也要守系统线程纪律。**

11.7 销毁路径：三路合流 bringDownServiceLocked

三个入口汇入同一个销毁函数（:4569）：显式 stopService/stopSelf（stopServiceLocked:1248）、最后一根绑定解开且不再 started（unbindServiceLocked:3150 → App 侧 onUnbind 返回 false → stop）、以及启动途中被叫停的补刀（11.4⑩）。

java frameworks/base/services/core/java/com/android/server/am/ActiveServices.java:4569-4700（节选+讲解注释）

```
private void bringDownServiceLocked(ServiceRecord r, boolean enqueueOomAdj) {
// 讲解注释①：通知所有绑定方"服务死了"：conn.connected(r.name, null,
// true) → 客户端 ServiceDispatcher 触发 onServiceDisconnected（参数
// null）。cr.serviceDead 置真，之后 binder 也不再转发。
ArrayMap<IBinder, ArrayList<ConnectionRecord>> connections = r.getConnections();
for (int conni = connections.size() - 1; conni >= 0; conni--) {
...
for (int i = 0; i < c.size(); i++) {
ConnectionRecord cr = c.get(i);
cr.serviceDead = true;
cr.stopAssociation();
...
cr.conn.connected(r.name, null, true);
...
}
}
boolean oomAdjusted = false;
// 讲解注释②：对每个"已绑定"的 IntentBindRecord 下发 onUnbind：
// scheduleUnbindService → App 侧 handleUnbindService（:4566）。
if (r.app != null && r.app.getThread() != null) {
for (int i = r.bindings.size() - 1; i >= 0; i--) {
IntentBindRecord ibr = r.bindings.valueAt(i);
if (ibr.hasBound) {
...
oomAdjusted |= bumpServiceExecutingLocked(r, false, "bring down unbind",
OomAdjuster.OOM_ADJ_REASON_UNBIND_SERVICE);
ibr.hasBound = false;
ibr.requested = false;
r.app.getThread().scheduleUnbindService(r, ibr.intent.getIntent());
...
}
}
}
// 讲解注释③：fgRequired 期间被销毁 → 不只是停，还要追责：
// 走 serviceForegroundCrash 抛 ForegroundServiceDidNotStartInTimeException
// （11.6 的第三行表格）。AppOps 的 OP_START_FOREGROUND 同步结账。
if (r.fgRequired) {
Slog.w(TAG_SERVICE, "Bringing down service while still waiting for start foreground: "
```

```
+ r);
r.fgRequired = false;
r.fgWaiting = false;
...
mAm.mHandler.removeMessages(
ActivityManagerService.SERVICE_FOREGROUND_TIMEOUT_MSG, r);
if (r.app != null) {
Message msg = mAm.mHandler.obtainMessage(
ActivityManagerService.SERVICE_FOREGROUND_CRASH_MSG);
...
}
}
...
// 讲解注释④：从 ServiceMap 摘除本体与 intent 索引；顺手清掉复活
// 机器（totalRestartCount 归零、unscheduleServiceRestartLocked）。
ServiceRecord found = smap.mServicesByInstanceName.remove(r.instanceName);
...
smap.mServicesByIntent.remove(r.intent);
r.totalRestartCount = 0;
unscheduleServiceRestartLocked(r, 0, true);
// 讲解注释⑤：从待兑现清单里除名——销毁先于孵化完成（调用方
// start 后立刻 stop 的竞态）也能正确收场。
for (int i = mPendingServices.size() - 1; i >= 0; i--) {
if (mPendingServices.get(i) == r) {
mPendingServices.remove(i);
...
}
}
...
// 讲解注释⑥：前台通知撤下、前台计数递减（decActiveForegroundAppLocked
// ——若这是进程最后一个前台服务，进程将失去 FGS 优先级背书，
// 下一轮 OomAdjuster 重算时会掉档，见第十四章）。
cancelForegroundNotificationLocked(r);
final boolean exitingFg = r.isForeground;
if (exitingFg) {
decActiveForegroundAppLocked(smap, r);
...
}
// 讲解注释⑦：对活着的宿主下发 onDestroy（scheduleDestroyService）。
...
```

逐段精读：①的"断连通知"与②的"unbind 通知"**是两件事**：onServiceDisconnected 告诉绑定方"服务没了"，onUnbind 告诉服务"没人绑你了"。第三章 3.2 说的"started/bound 两本账独立，双清才销毁"在销毁函数里看得最清楚：只有 started 账（stopService）与 bound 账（全部 unbind 且 onUnbind 返回 false）都清零，`r.startRequested` 与 `r.bindings` 同时空，销毁才是终态——否则只是部分结账。

11.8 复活机器：三种意愿 × 指数退避

进程被杀后，AMS 依据 onStartCommand 的返回值决定是否重启（`scheduleServiceRestartLocked:3685` / `performScheduleRestartLocked:3857`），但**加了退避与配额**（`ActivityManagerConstants:168-170`）：首次延迟 `SERVICE_RESTART_DURATION`（默认 1s），每次

失败乘 SERVICE_RESTART_DURATION_FACTOR（默认 4，1s→4s→16s→64s…上封顶 1 小时）；服务若已连续运行超过 SERVICE_RESET_RUN_DURATION（默认 60s），退避序列重置。另有两个"不再复活"的条件：stopSelfResult(startId) 与最新 startId 匹配（自己宣布使命完成）、App 被用户强制停止（force-stop 清光一切记账，复活机器一并熄火）。

- **START_NOT_STICKY**：不自动补发 pendingStarts——"欠条随进程死亡作废"。
- **START_STICKY**：欠条作废但**伪造无参欠条**（11.4⑧）——重启必到，payload 不保证。
- **START_REDELIVER_INTENT**：未完成的 StartItem 连原 intent 重投（START_FLAG_REDELIVERY）——payload 必达。

设计洞察 · 逐段精读：退避是给"杀-启死循环"装刹车

没有退避的话，一个启动即崩的服务会以秒级频率拉起 zygote fork——一次内存压力事件就能变成 fork 风暴。1×4^n 的指数退避把最坏情况的启动频率压到可忽略，同时保住"正常服务快速恢复"的体验（首次仅 1s）。**框架从不裸实现用户可见语义，永远外挂一层系统自保护**——这条规律在 onStartCommand 返回值、广播重投、Provider 重启计数（第十三章 13.6 的 MAX_RETRY_COUNT）上三度重现。

源码锚点 · 第十一章锚点速查

ActiveServices.java:866 startServiceInnerLocked（fgRequired/延迟启动）/ :4148 bringUpServiceLocked / :4332 realStartServiceLocked / :4478 fgRequired 上弦 / :4569 bringDownServiceLocked
:3644 executingStart 记账 / :5712 serviceTimeout / :5763 serviceForegroundTimeout / :5818 serviceForegroundCrash / :5827 scheduleServiceTimeoutLocked
ActivityThread.java:4477 handleCreateService / :4673 handleServiceArgs / ActivityManagerConstants.java:168-170 退避常量 / :231 30s 追责窗口

第十二章 Broadcast 深水区：动态注册桥、有序队列与超时机器

设计洞察 · 学习目标

1. 走通一条动态广播从 registerReceiver 到主线程 onReceive 的七步旅程；2. 解释 ReceiverDispatcher/InnerReceiver 双层结构与"动态接收者永远在注册线程回调"的机制根源；3. 背出 broadcastTimeoutLocked 的五道豁免与判决动作；4. 说明 T 上动态接收者导出标志新规、粘性广播处境与 BOOT_COMPLETED 延迟三件事

第四章给出了双队列流水线的骨架（processNextBroadcastLocked 四段），本章补齐上游（注册桥）、下游（超时判决书全文）与 T 上的新变量（导出标志、粘性广播、BOOT_COMPLETED 延迟）。

读完本章，dumpsys activity broadcasts 里的 BroadcastRecord/BroadcastFilter/ReceiverList 三层结构就都是熟脸了。

12.1 一条动态广播的完整旅程（七步）

1. **注册**：registerReceiver → LoadedApk 为（Context, Receiver）组合查/建 ReceiverDispatcher（LoadedApk.java:1588），取出其 binder 桩 InnerReceiver 交给 AMS。

2. **AMS 侧闸门**：ActivityManagerService.registerReceiver（:13043）——导出标志检查（T 新规）、粘性匹配、ReceiverList/mRegisteredReceivers 记账，返回粘性 Intent。

3. **发送**：sendBroadcast → broadcastIntent → broadcastIntentLocked 判定接收者名单（动态在前、manifest 在后），按调用方前后台**入不同队列**：enqueueParallelBroadcastLocked（:262）或 enqueueOrderedBroadcastLocked（:270）。

4. **驱动**：scheduleBroadcastsLocked（:468）投一条 BROADCAST_INTENT_MSG（mBroadcastsScheduled 防重入）→ processNextBroadcast（:1137）→ processNextBroadcastLocked（:1149）。

5. **并行段出货**：while 循环逐条 deliverToRegisteredReceiverLocked（:685）→ performReceiveLocked（:640）——每个接收者一个 Binder 调用，不等回执。

6. **有序段推进**：取队首当前 receiver → 进程在：processCurBroadcastLocked（:323）下发并上弦超时；进程不在：孵化并挂 mPendingBroadcast（第四章第二段的"欠条"，BroadcastQueue.java:1913）。

7. **回调落地**：Binder 进 App → InnerReceiver.performReceive → ReceiverDispatcher 的 Handler post → Args.getRunnable() → **注册时指定的线程**上 onReceive → finishReceiver 回执，有序队列才走下一位。

12.2 registerReceiver：AMS 侧的闸门（T 新规现场）

java frameworks/base/services/core/java/com/android/server/am/ActivityManagerService.java:13043-13199（节选+讲解注释）

```
public Intent registerReceiver(IApplicationThread caller, String callerPackage,
..., IIntentReceiver receiver, IntentFilter filter, ..., int flags, ...) {
...
// 讲解注释①：T 的新规判据：接收者若不是"只收受保护广播"
// （onlyProtectedBroadcasts），必须显式声明导出意图。
final boolean explicitExportStateDefined =
(flags & (Context.RECEIVER_EXPORTED | Context.RECEIVER_NOT_EXPORTED)) != 0;
if (((flags & Context.RECEIVER_EXPORTED) != 0) && (
(flags & Context.RECEIVER_NOT_EXPORTED) != 0)) {
throw new IllegalArgumentException(
"Receiver can't specify both RECEIVER_EXPORTED and RECEIVER_NOT_EXPORTED"
+ "flag"); // 讲解注释②：两个标志互斥，同时给直接抛。
}
...
```

```
// 讲解注释③：requireExplicitFlagForDynamicReceivers 按 targetSdk 的
// 兼容开关（DYNAMIC_RECEIVER_EXPLICIT_EXPORT_REQUIRED）生效：T 以上
// 不声明 → SecurityException；T 以下维持旧行为（默认导出）。
final boolean requireExplicitFlagForDynamicReceivers = CompatChanges.isChangeEnabled(
DYNAMIC_RECEIVER_EXPLICIT_EXPORT_REQUIRED, callingUid);
if (!onlyProtectedBroadcasts) {
if (receiver == null && !explicitExportStateDefined) {
// 讲解注释④：receiver==null 是"粘性查询"语义（不注册，只要
// 粘性值）——框架放行并默认 RECEIVER_EXPORTED。
flags |= Context.RECEIVER_EXPORTED;
} else if (requireExplicitFlagForDynamicReceivers && !explicitExportStateDefined) {
throw new SecurityException(
callerPackage + ": One of RECEIVER_EXPORTED or "
+ "RECEIVER_NOT_EXPORTED should be specified when a receiver "
+ "isn't being registered exclusively for system broadcasts");
} else if (!requireExplicitFlagForDynamicReceivers && (
(flags & Context.RECEIVER_NOT_EXPORTED) == 0)) {
flags |= Context.RECEIVER_EXPORTED;
}
}
...
// 讲解注释⑤：粘性匹配：拿 filter 去 mStickyBroadcasts（按 action 分桶
// 的粘性 Intent 库，AMS:1099）里逐条 match，命中的收集进 allSticky。
ArrayList<Intent> allSticky = null;
if (stickyIntents != null) {
...
for (int i = 0, N = stickyIntents.size(); i < N; i++) {
Intent intent = stickyIntents.get(i);
...
if (filter.match(resolver, intent, true, TAG) >= 0) {
if (allSticky == null) {
allSticky = new ArrayList<Intent>();
}
allSticky.add(intent);
}
}
}
// 讲解注释⑥：第一条粘性值直接作为返回值交回调用方（马上可用）；
// 后续粘性值按普通广播投递给这个新注册的接收者。
Intent sticky = allSticky != null ? allSticky.get(0) : null;
if (receiver == null) {
return sticky; // 讲解注释⑦：纯粘性查询：不建 ReceiverList。
}
// 讲解注释⑧：正式注册：ReceiverList 按 binder 挂进 mRegisteredReceivers
//（进程死亡时整表回收——书3 死亡通知的又一位客户），粘性值立即投递。
...
```

逐段精读：③④⑤合起来就是"粘性广播在 T 的处境"的精确表述：**机制仍在**（查询语义完整保留，注释里还写着"Sticky broadcasts should not be used generally"），**但没有被硬禁**——真正的收紧发生在注册面（导出标志）而非粘性面。粘性广播的本质缺陷在源码注释里一目了然：它把"最后一次状态"与"任意进程都能读的公开账本"混为一谈，权限、过期、清理（AMS:4511 按用户清理）全都要系统兜底。现代替代是"显式状态源 + 动态注册实时刷新"（如 ActionProvider/registerReceiver 监听系统已收窄的白名单动作）。

12.3 ReceiverDispatcher：App 侧的桥

java frameworks/base/core/java/android/app/LoadedApk.java:1868-1911（节选+讲解注释）

```
// InnerReceiver（IIntentReceiver.Stub）→ ReceiverDispatcher.performReceive
public void performReceive(Intent intent, int resultCode, String data,
Bundle extras, boolean ordered, boolean sticky, int sendingUser) {
// 讲解注释①：把这次投递打包成 Args——它继承自
// BroadcastReceiver.PendingResult：finish() 就是 goAsync() 持有的
// 那张"延时回执单"，有序广播的 finishReceiver 全靠它回系统。
final Args args = new Args(intent, resultCode, data, extras, ordered,
sticky, sendingUser);
if (intent == null) {
Log.wtf(TAG, "Null intent received");
} else {
...
}
// 讲解注释②：post 到注册时绑定的 Handler（默认主线程）——
// "动态接收者永远在注册线程回调"的机制根源就在这一个 post。
if (intent == null || !mActivityThread.post(args.getRunnable())) {
// 讲解注释③：post 失败（Looper 退出/intent 为空）→ 若是有序
// 广播且还注册着，**代收**：替接收者立刻 sendFinished，
// 有序链不被一个死掉的 Looper 卡死。
if (mRegistered && ordered) {
IActivityManager mgr = ActivityManager.getService();
...
args.sendFinished(mgr);
}
}
}
```

逐段精读：③是**自愈设计**的教科书案例——注册桥死后，系统侧的有序队列不应陪葬。`Args.getRunnable()` 内部还有一层对称处理：接收者对象若已被 unregister（`mDispatcher` 查不到），走 InnerReceiver 的外层分支（`LoadedApk.java:1700`）以"未注册接收者"的身份 `mgr.finishReceiver(...)` 代收回执。**回执永远有人负责**——这条不变量撑起了有序广播的有限步终止性。

12.4 双队列与超时上弦

队列实例在 AMS 构造时定装（`ActivityManagerService.java:2410-2417`）：`foreground`（`foreConstants`，不延迟）与 `background`（`backConstants`，延迟投递）为主，T 另有 `offload_bg/offload_fg` 两条卸载队列（默认常量下行为与主队列一致，SLOW_TIME 关闭）。**前台 10s / 后台 60s 的差异不在 BroadcastConstants 的默认值，而在 AMS 显式覆盖**（`:2398-2405`）：`foreConstants.TIMEOUT = BROADCAST_FG_TIMEOUT`（10s，`:516`）、`backConstants.TIMEOUT = BROADCAST_BG_TIMEOUT`（60s，`:517`）。

有序广播每次投递一个 receiver 都校一次闹钟：

java frameworks/base/services/core/java/com/android/server/am/BroadcastQueue.java:1481-1507（节选+讲解注释）

```
if (! mPendingBroadcastTimeoutMessage) {
```

```
// 讲解注释①：只有当前没有挂着的超时消息时才设新闹钟——超时
// 是"接力式"的：每个 receiver 的窗口从它的投递时刻起算
// （receiverTime + mConstants.TIMEOUT），不是全广播共享一个大钟。
long timeoutTime = r.receiverTime + mConstants.TIMEOUT;
if (DEBUG_BROADCAST) Slog.v(TAG_BROADCAST,
"Submitting BROADCAST_TIMEOUT_MSG ["
+ mQueueName + "] for " + r + " at " + timeoutTime);
setBroadcastTimeoutLocked(timeoutTime);
}
...
if (nextReceiver instanceof BroadcastFilter) {
// 讲解注释②：动态接收者：直接 Binder 下发（performReceiveLocked）。
BroadcastFilter filter = (BroadcastFilter)nextReceiver;
...
deliverToRegisteredReceiverLocked(r, filter, r.ordered, recIdx);
if (r.receiver == null || !r.ordered) {
// 讲解注释③：快速完结——receiver 在投递瞬间就已 finish
//（例如无序广播或跳过），立刻推进下一位，不等回执。
r.state = BroadcastRecord.IDLE;
scheduleBroadcastsLocked();
}
return;
}
// 讲解注释④：manifest 接收者：可能要孵化进程（12.1 第 6 步的
// "欠条"分支，BroadcastQueue.java:1888-1914）。
```

12.5 broadcastTimeoutLocked：判决书全文

java
frameworks/base/services/core/java/com/android/server/am/BroadcastQueue.java:2068-2159（节选+讲解注释）

```
final void broadcastTimeoutLocked(boolean fromMsg) {
if (fromMsg) {
mPendingBroadcastTimeoutMessage = false;
}
if (mDispatcher.isEmpty() || mDispatcher.getActiveBroadcastLocked() == null) {
return; // 讲解注释①：队列已空/活动广播不在 → 迟到的闹钟，无罪。
}
long now = SystemClock.uptimeMillis();
BroadcastRecord r = mDispatcher.getActiveBroadcastLocked();
if (fromMsg) {
if (!mService.mProcessesReady) {
return; // 讲解注释②：系统未就绪期（开机早期）不判（冤案）。
}
if (r.timeoutExempt) {
return; // 讲解注释③：豁免广播直接放过。本树上仅
// ACTION_PRE_BOOT_COMPLETED 被豁免（AMS:13995）——
// 它的开机全量回放可能要数分钟，不该按 10s 判。
}
long timeoutTime = r.receiverTime + mConstants.TIMEOUT;
if (timeoutTime > now) {
// 讲解注释④：**过早超时**：闹钟是接力式的，本 receiver 提前
// finish 后下一个 receiver 的窗口还没到 → 不重设大钟，
// 只把闹钟顺延到新的到期时刻。
setBroadcastTimeoutLocked(timeoutTime);
```

```
return;
}
}
if (r.state == BroadcastRecord.WAITING_SERVICES) {
// 讲解注释⑤：广播已投完、在等 startService 收尾 → 等够就推进（不 ANR）。
Slog.i(TAG, "Waited long enough for: " + ...);
r.curComponent = null;
r.state = BroadcastRecord.IDLE;
processNextBroadcastLocked(false, false);
return;
}
// 讲解注释⑥：豁免调试进程——断点不是罪（与 serviceTimeout ①同款）。
final boolean debugging = (r.curApp != null && r.curApp.isDebugging());
Slog.w(TAG, "Timeout of broadcast " + r + " - receiver=" + r.receiver
+ ", started " + (now - r.receiverTime) + "ms ago");
r.receiverTime = now;
if (!debugging) {
r.anrCount++;
}
...
// 讲解注释⑦：定位嫌疑进程：动态接收者按 receiverList.pid 查
// mPidsSelfLocked；manifest 接收者用 r.curApp。命中即拼 ANR 文案：
if (app != null) {
anrMessage = "Broadcast of " + r.intent.toString();
}
...
// 讲解注释⑧：**先推进、后追责**：把本 receiver 记为已完结
//（finishReceiverLocked），调度下一位——广播链继续走，烂的只是这一环
//（第四章第四段"单点故障不扩散"的源码现场）。
finishReceiverLocked(r, r.resultCode, r.resultData,
r.resultExtras, r.resultAbort, false);
scheduleBroadcastsLocked();
if (!debugging && anrMessage != null) {
mService.mAnrHelper.appNotResponding(app, anrMessage);
}
}
```

逐段精读：logcat 里 ANR 行的 `anrMessage` 就是 ⑦⑧ 拼出来的 *"Broadcast of Intent { act=... }"*——看到它先别慌，它只说明**这个接收者**卡了 10s（前台）/60s（后台），整条广播链在⑧里已经继续推进了。⑤的 WAITING_SERVICES 是 T 上一个容易被误诊的状态：有序广播完成后系统会等 `startService`（onReceive 里 startService 的常见写法）收敛，这段等待超时**不产生 ANR**，只放弃等待。

12.6 T 上的三个新变量

- **导出标志新规**：12.2③——动态接收者不再默认"只受保护广播保护"，注册非保护广播必须显式 `RECEIVER_EXPORTED/NOT_EXPORTED`。安全收益是收窄了"任何 App 都能给你发广播"的攻击面（配合 manifest 接收者的 `android:exported` 同款收紧）。
- **BOOT_COMPLETED 延迟**：`BroadcastConstants` 的兼容开关 `DEFER_BOOT_COMPLETED_BROADCAST_CHANGE_ID`（203704822，targetSdk > S_V2 默认启用，：68-88）——LOCKED_BOOT_COMPLETED/BOOT_COMPLETED **推迟到该 UID 首个进程启动**才投

递（`BroadcastQueue.updateUidReadyForBootCompletedBroadcastLocked:394`），配合 `updateOomAdjLSP` 的进程孵化节奏，避免开机瞬间被 30 个 receiver 各拉起一个进程。这是"粘性广播式开机状态"的现代替代之一。

- **缓存/冻结进程绕行**：`deliverToRegisteredReceiverLocked`（:685）开头就是一串 skip 判定：association 校验、IntentFirewall、所需权限、进程已死、instant app 可见性……命中即跳过该接收者（记录 DELIVERY_SKIPPED）。配合第十四章的冻结机制：冻结进程的动态广播默认 deferred，解冻补投——**"缓存接收者不拖累投递链"** 是双队列之外的第二层隔离。

> **设计洞察 · 逐段精读：为什么有序广播的超时线比广播"总时长"语义更严**
>
> 超时按 receiver 逐个接力（12.4①），意味着 N 个接收者最坏要 N × 10s——这不是缺陷而是**责任隔离**：每个接收者只为自己那 10s 负责，上一个慢接收者不会吃掉下一个的窗口。对比 ANR 家族里 Service 的"每次回调 20s"、输入的"每次事件 5s"，ANR 的计时单位永远是**单次调度步**而非整个业务流程——理解了这一点，你就掌握了 Android 超时语义的通则。

> **源码锚点 · 第十二章锚点速查**
>
> ActivityManagerService.java:13043 registerReceiver / :516-517 BROADCAST_FG/BG_TIMEOUT / :2410-2417 四队列定装
LoadedApk.java:1588/1668/1868 getReceiverDispatcher/ReceiverDispatcher/performReceive
BroadcastQueue.java:262/270/468/640/685/1149/1481/1913/2053/2068 注册→分发→判决全链

第十三章 ContentProvider 发布-等待-回收：三条链条的合流

> **设计洞察 · 学习目标**
>
> 1. 走通获取/发布/死亡三条链并指出 mLaunchingProviders 在每条链上的记账动作；2. 区分 T 上 App 调用方（客户端 ProviderKey 等待）与外部调用方（AMS 侧 cpr.wait）的两条等待路径；3. 说明 stable/unstable 引用计数与"5 秒延迟回收"的经济学；4. 复述进程死亡时 removeDyingProviderLocked 对稳定/不稳定客户端的差别处置

第五章 5.1 给出了 `getContentProviderImpl` 的骨架与"同步阻塞入口"的定性，本章把三条链条（获取-等待、发布-兑现、死亡-清算）逐段展开到源码级，并修正一个常见的过时表述：**T 上 App 调用方的等待已从 AMS 大锁内挪到了客户端侧**——这正是把 5.1 的骨架读"活"的关键一步。

13.1 三条链条总览

- **获取链**（`getContentProviderImpl`，`ContentProviderHelper.java:158`）：按 authority 查 `providerMap` → 宿主进程在：直接建 `ContentProviderConnection` 返回；不在：孵化 +

mLaunchingProviders 记账 + conn.waiting = true。

- **发布链**（publishContentProviders，:636）：宿主进程 attach 后一揽子登记 → 从 mLaunchingProviders 摘除 → 撤销超时消息 → notifyAll + 逐连接通知等待方（T 的新姿势）。
- **死亡链**（removeDyingProviderLocked，:1695）：宿主死亡 → 在发布中的记重试（上限 MAX_RETRY_COUNT）；已发布的把连接标记 dead → **稳定客户端直接陪葬**（killLocked），不稳定客户端收到 unstableProviderDied 回调自行善后。

三条链交汇在同一个数据结构 ContentProviderRecord（cpr）：provider（IContentProvider binder，null 即"未发布"）、launchingApp（正在孵化的宿主）、connections（引用计数账）、mRestartCount（死亡重试计数）。

13.2 mLaunchingProviders：孵化决策与记账

java frameworks/base/services/core/java/com/android/server/am/ContentProviderHelper.java:431-521（节选+讲解注释）

```
// 讲解注释①：先查"发布中"清单：同一条 provider 是否已有人
// 在孵化？mLaunchingProviders 是全局单例清单（:100），
// 防止两个调用方同时为同一 provider 起两个进程。
final int numLaunchingProviders = mLaunchingProviders.size();
int i;
for (i = 0; i < numLaunchingProviders; i++) {
if (mLaunchingProviders.get(i) == cpr) {
break;
}
}
if (i >= numLaunchingProviders) {
final long origId = Binder.clearCallingIdentity();
try {
...
// 讲解注释②：provider 即将被用 → 宿主包摘掉 stopped
// 帽子（与 Service 11.3⑤ 同款反向记账）。
AppGlobals.getPackageManager().setPackageStoppedState(
cpr.appInfo.packageName, false, userId);
...
// 讲解注释③：宿主进程存在且活着 → 不用孵化，直接
// scheduleInstallProvider 让宿主补发布即可。
ProcessRecord proc = mService.getProcessRecordLocked(
cpi.processName, cpr.appInfo.uid);
IApplicationThread thread;
if (proc != null && (thread = proc.getThread()) != null
&& !proc.isKilled()) {
...
if (!pr.hasProvider(cpi.name)) {
pr.installProvider(cpi.name, cpr);
try {
thread.scheduleInstallProvider(cpi);
} catch (RemoteException e) {
}
}
} else {
// 讲解注释④：宿主不在 → 孵化。HostingRecord 记
```

```
// HOSTING_TYPE_CONTENT_PROVIDER——进程出生证明上
// 写着"为 provider 而生"，后续优先级核算可见。
proc = mService.startProcessLocked(
cpi.processName, cpr.appInfo, false, 0,
new HostingRecord(HostingRecord.HOSTING_TYPE_CONTENT_PROVIDER,
new ComponentName(...)),
Process.ZYGOTE_POLICY_FLAG_EMPTY, false, false);
...
}
cpr.launchingApp = proc;
// 讲解注释⑤：记账入"发布中"清单——13.1 获取链的
// 核心动作；此后所有后来者走①的短路（不重复孵化）。
mLaunchingProviders.add(cpr);
} finally {
Binder.restoreCallingIdentity(origId);
}
}
// 讲解注释⑥：providerMap 双索引落账（byClass + byName，
// authority 可分号拆多个）。引用计数开账，并标 waiting。
if (firstClass) {
mProviderMap.putProviderByClass(comp, cpr);
}
mProviderMap.putProviderByName(name, cpr);
conn = incProviderCountLocked(r, cpr, token, callingUid, callingPackage,
callingTag, stable, false, startTime, mService.mProcessList,
expectedUserId);
if (conn != null) {
conn.waiting = true;
}
```

逐段精读：①⑤是一对"**先查账、后记账**"——mLaunchingProviders 本质是一张**孵化去重表**。注意③的第三种情况：宿主进程活着但 provider 尚未发布（冷启动竞态），此时**不孵化也不等待**，直接让宿主补发布（scheduleInstallProvider）——Provider 的调度空间其实是三态而非两态，这是读 5.1 骨架时最容易漏的一格。

13.3 两条等待路径：T 上的精确图景

5.1 写过"调用方线程持锁等待（`mLaunchingProviders` 记账），`publishContentProviders` 唤醒"——这是对外部调用方（shell、dumpsys 等 `caller == null` 的场景）依然正确的描述。但 T 上 **App 调用方已经不在 AMS 里等了**：AMS 立即返回一个 `provider == null` 的 holder，等待发生在客户端 `ActivityThread.acquireProvider`。

java frameworks/base/core/java/android/app/ActivityThread.java:7029-7051（节选+讲解注释）

```
ContentProviderHolder holder = null;
final ProviderKey key = getGetProviderKey(auth, userId);
try {
synchronized (key) {
holder = ActivityManager.getService().getContentProvider(
getApplicationThread(), c.getOpPackageName(), auth, userId, stable);
// 讲解注释①：holder 非空但 provider 为空且非本地 → 进入等待。
if (holder != null && holder.provider == null && !holder.mLocal) {
synchronized (key.mLock) {
```

```
if (key.mHolder != null) {
// 讲解注释②：同 authority 的并发请求已有赢家拿到
// 结果——直接复用（ProviderKey 按 auth+userId 缓存）。
} else {
// 讲解注释③：等 CONTENT_PROVIDER_READY_TIMEOUT_MILLIS
// （= PUBLISH 超时 10s + 10s，ContentResolver:794）。
// 调用方线程在此休眠——query() 的同步语义由
// 客户端自己扛，不再占用 system_server 的大锁。
key.mLock.wait(ContentResolver.CONTENT_PROVIDER_READY_TIMEOUT_MILLIS);
}
holder = key.mHolder;
}
if (holder != null && holder.provider == null) {
// 讲解注释④：醒来仍为 null → 超时，按失败处理。
holder = null;
}
}
}
}
```

唤醒方有两个：**好结局**——AMS 发布成功后回调客户端 `notifyContentProviderPublishStatus`（`ActivityThread.java:1888`，`key.mHolder = holder; key.mLock.notifyAll()`），其上游是 `ContentProviderRecord.onProviderPublishStatusLocked`（`ContentProviderRecord.java:217`，遍历 waiting 连接逐个通知）；**坏结局**——宿主死亡，`removeDyingProviderLocked` 以 `status=false` 走同一条通知链，客户端醒来拿到 null，抛出你熟悉的 *"Failed to find provider info"*。

等待路径	谁在等	等待位置	唤醒方
App 调用方（caller 非 null）	调用 App 的发起线程	ActivityThread ProviderKey.mLock.wait	notifyContentProviderPublishStatus
外部调用方（shell/dumpsys 等）	system_server 内 binder 线程	ContentProviderHelper cpr.wait 循环（:549-570）	cpr.notifyAll（publish 或 removeDying）

设计洞察 · 设计洞察：等待为什么要搬家

旧版在 AMS 大锁内 `cpr.wait`，意味着**一个慢 provider 能把 system_server 的调度中枢按住**——所有 AMS 请求陪等。搬到客户端后，system_server 只做记账与一次性通知（"以后别等我，好了叫你"），等待成本转移给唯一的利益相关方。这是"**同步语义保留给调用者，异步实现留给系统**"的经典折中——与第六章"Provider 语义上就要返回值、无法欠条化"的定性并不矛盾：同步的语义没变，同步的**位置**挪了。

13.4 publishContentProviders：一揽子兑现

java frameworks/base/services/core/java/com/android/server/am/ContentProviderHelper.java:636-712（节选+讲解注释）

```
void publishContentProviders(IApplicationThread caller, List<ContentProviderHolder>
providers) {
...
synchronized (mService) {
// 讲解注释①：验明发布者：必须是登记在册的进程（防伪造发布），
// 且不得是 isolated/sdkSandbox 进程（enforceNotIsolatedOrSdkSandbox）。
```

```
final ProcessRecord r = mService.getRecordForAppLOSP(caller);
...
if (r == null) {
throw new SecurityException("Unable to find app for caller ...");
}
final long origId = Binder.clearCallingIdentity();
boolean providersPublished = false;
for (int i = 0, size = providers.size(); i < size; i++) {
ContentProviderHolder src = providers.get(i);
...
// 讲解注释②：按 name 找到启动时预生成的 ContentProviderRecord
// （generateApplicationProvidersLocked 在 attach 前已按 manifest
// 建好 cpr——这里只是把"骨架"填上"肉"：真正的 binder）。
ContentProviderRecord dst = r.mProviders.getProvider(src.info.name);
if (dst == null) {
continue;
}
providersPublished = true;
// 讲解注释③：落进 providerMap 双索引（authority 分号拆分）。
mProviderMap.putProviderByClass(comp, dst);
String[] names = dst.info.authority.split(";");
for (int j = 0; j < names.length; j++) {
mProviderMap.putProviderByName(names[j], dst);
}
// 讲解注释④：从"发布中"清单摘除 + 撤两张超时消息
// （WAIT_FOR_CONTENT_PROVIDER_TIMEOUT_MSG 与
// CONTENT_PROVIDER_PUBLISH_TIMEOUT_MSG——后者到点会杀宿主，
// 见 13.7）。
boolean wasInLaunchingProviders = false;
for (int j = 0, numLaunching = mLaunchingProviders.size(); j < numLaunching; j++) {
if (mLaunchingProviders.get(j) == dst) {
mLaunchingProviders.remove(j);
wasInLaunchingProviders = true;
...
}
}
if (wasInLaunchingProviders) {
mService.mHandler.removeMessages(
ActivityManagerService.WAIT_FOR_CONTENT_PROVIDER_TIMEOUT_MSG, dst);
mService.mHandler.removeMessages(
ActivityManagerService.CONTENT_PROVIDER_PUBLISH_TIMEOUT_MSG, r);
}
...
synchronized (dst) {
// 讲解注释⑤：填肉：provider binder 就位；notifyAll 唤醒
// AMS 侧的外部等待者；再逐连接通知客户端等待者（13.3）。
dst.provider = src.provider;
dst.setProcess(r);
dst.notifyAll();
dst.onProviderPublishStatusLocked(true);
}
dst.mRestartCount = 0; // 讲解注释⑥：死亡重试计数清零。
...
}
// 讲解注释⑦：发布改变进程价值 → 立刻重算 OomAdj
```

```
// (OOM_ADJ_REASON_GET_PROVIDER：宿主因 hosting provider 换档)。
if (providersPublished) {
mService.updateOomAdjLocked(r, OomAdjuster.OOM_ADJ_REASON_GET_PROVIDER);
...
}
Binder.restoreCallingIdentity(origId);
}
```

逐段精读：②⑤合起来解释了 5.2 的"发布是一揽子的"：manifest 声明的全部 provider 在 attach 前已由 `generateApplicationProvidersLocked` 生成 cpr 骨架，attach 后一次 publish 全部填肉——**不存在"半个 provider 可见"的中间态**。⑦是发布链的收尾动作：宿主进程的账面价值变了（"它现在养着 provider"），OomAdjuster 立刻重算——这条线直通第十四章。

13.5 引用计数经济学：inc/dec 与 5 秒延迟回收

java frameworks/base/services/core/java/com/android/server/am/ContentProviderHelper.java:1357-1436（节选+讲解注释）

```
private ContentProviderConnection incProviderCountLocked(ProcessRecord r,
final ContentProviderRecord cpr, IBinder externalProcessToken, int callingUid,
String callingPackage, String callingTag, boolean stable, boolean updateLru,
long startTime, ProcessList processList, @UserIdInt int expectedUserId) {
if (r == null) {
// 讲解注释①：外部调用方（无 ProcessRecord，如 shell）→ 用
// externalProcessToken 记外部句柄账，不建 Connection。
cpr.addExternalProcessHandleLocked(externalProcessToken, callingUid, callingTag);
return null;
}
final ProcessProviderRecord pr = r.mProviders;
for (int i = 0, size = pr.numberOfProviderConnections(); i < size; i++) {
ContentProviderConnection conn = pr.getProviderConnectionAt(i);
if (conn.provider == cpr) {
// 讲解注释②：同进程对同一 provider 的第 N 次访问 → 只递增
// 计数（stable/unstable 两个独立计数），不新建连接对象。
conn.incrementCount(stable);
return conn;
}
}
// 讲解注释③：首访：建 ContentProviderConnection，宿主进程因此获得
// HOSTING_COMPONENT_TYPE_PROVIDER 标记（影响 mProfile 与回收策略）。
ContentProviderConnection conn = new ContentProviderConnection(cpr, r, callingPackage,
expectedUserId);
...
if (updateLru && cpr.proc != null
&& r.mState.getSetAdj() <= ProcessList.PERCEPTIBLE_LOW_APP_ADJ) {
// 讲解注释④：可感知客户端在访问 → 宿主在 LRU 链上"回炉"往后排
//（不容易被当最旧缓存杀掉）。provider 启动贵，值得这份待遇。
processList.updateLruProcessLocked(cpr.proc, false, null);
}
return conn;
}
private boolean decProviderCountLocked(ContentProviderConnection conn,
ContentProviderRecord cpr, IBinder externalProcessToken, boolean stable,
boolean enforceDelay, boolean updateOomAdj) {
```

```
...
if (conn.totalRefCount() > 1) {
conn.decrementCount(stable);
return false; // 讲解注释⑤：还有别的引用 → 只减数，不拆连接。
}
if (enforceDelay) {
// 讲解注释⑥：最后一根引用解除 → 延迟 5 秒再真拆——
// "释放后又立刻重取"的高频模式（列表页滑出又滑回）不再抖动。
BackgroundThread.getHandler().postDelayed(() -> {
handleProviderRemoval(conn, stable, updateOomAdj);
}, 5 * 1000);
} else {
handleProviderRemoval(conn, stable, updateOomAdj);
}
return true;
}
```

逐段精读：②④⑥合起来是一套**进程保活经济学**：`stable` 引用（acquireProvider 的 stable 参数，对应 `ContentResolver.acquireProvider` 语义）在宿主死亡时会把客户端一起带走（13.6），所以客户端不敢滥用；`unstable` 引用便宜但随时可能收到死亡回调。5 秒延迟回收则消灭了"解绑-重绑抖动"——**高频小操作的最优化不在让单次更快，而在让重复变免费**。这个 5s 常量没有名字，但它守护的是整个 ContentProvider 生态的访问模式（浏览、查询、再浏览）。

13.6 进程死亡清算：removeDyingProviderLocked

java frameworks/base/services/core/java/com/android/server/am/ContentProviderHelper.java:1695-1785（节选+讲解注释）

```
boolean removeDyingProviderLocked(ProcessRecord proc, ContentProviderRecord cpr,
boolean always) {
boolean inLaunching = mLaunchingProviders.contains(cpr);
if (inLaunching && !always && ++cpr.mRestartCount > ContentProviderRecord.MAX_RETRY_COUNT) {
// 讲解注释①：发布中反复死亡 → 超过 MAX_RETRY_COUNT 就放弃重启，
// 直接按"已发布又死亡"处理（不再给第三次机会）。
always = true;
}
if (!inLaunching || always) {
synchronized (cpr) {
cpr.launchingApp = null;
cpr.notifyAll(); // 讲解注释②：唤醒 AMS 侧外部等待者。
cpr.onProviderPublishStatusLocked(false);
// 讲解注释③：以 status=false 通知客户端等待者 → 客户端
// 醒来拿到 null → "Failed to find provider info"。
mService.mHandler.removeMessages(
ActivityManagerService.WAIT_FOR_CONTENT_PROVIDER_TIMEOUT_MSG, cpr);
}
...
// 讲解注释④：从 providerMap 双索引摘除（仅当还是"这一条"——
// 可能有新宿主已顶上，别把新人错杀）。
if (mProviderMap.getProviderByClass(cpr.name, userId) == cpr) {
mProviderMap.removeProviderByClass(cpr.name, userId);
}
...
}
```

```
for (int i = cpr.connections.size() - 1; i >= 0; i--) {
ContentProviderConnection conn = cpr.connections.get(i);
if (conn.waiting) {
if (inLaunching && !always) {
// 讲解注释⑤：还在等待中的连接先不动——宿主可能重启
// （inLaunching 且未超重试上限），继续等下一轮发布。
continue;
}
}
ProcessRecord capp = conn.client;
...
conn.dead = true;
if (conn.stableCount() > 0) {
// 讲解注释⑥：**稳定引用的宿主死了 → 客户端陪葬**。语义：
// 你 acquireProvider 声明"我依赖它稳定存在"，承诺由系统
// 强制兑现——这是 ContentProvider 体系里最重的契约。
final int pid = capp.getPid();
if (!capp.isPersistent() && thread != null
&& pid != 0 && pid != ActivityManagerService.MY_PID) {
capp.killLocked(
"depends on provider " + cpr.name.flattenToShortString()
+ " in dying proc " + (proc != null ? proc.processName : "??")
+ " (adj " + (proc != null ? proc.mState.getSetAdj() : "??") + ")",
ApplicationExitInfo.REASON_DEPENDENCY_DIED,
ApplicationExitInfo.SUBREASON_UNKNOWN,
true);
}
} else if (thread != null && conn.provider.provider != null) {
// 讲解注释⑦：不稳定引用 → 只回调 unstableProviderDied，
// 客户端自行 release+重取；系统替它拆连接、清 association。
thread.unstableProviderDied(conn.provider.provider.asBinder());
...
}
}
if (inLaunching && always) {
mLaunchingProviders.remove(cpr);
cpr.mRestartCount = 0;
inLaunching = false;
}
return inLaunching;
}
```

逐段精读：⑥⑦是"**契约分层**"的活教材：stable 客户端付的是"生死与共"的保费（数据一致性优先，客户端进程直接杀掉重来，避免它拿着失效假设继续运行）；unstable 客户端付的是"随时搬家"的折价（收到通知自己善后）。`ApplicationExitInfo.REASON_DEPENDENCY_DIED` 就是⑥留下的尸检记录——`dumpsys activity exit-info` 能直接读到"depends on provider ..."这句话。

13.7 超时三表与观测

超时	时长	到点动作	常量位置
发布超时（宿主没 publish）	10s	杀宿主进程（"Checking for process ... timeout waiting for provider"）	ContentResolver:786
就绪超时（客户端等待上限）	10s+10s	客户端醒来按失败处理	ContentResolver:794
getType 异步等待	3s	getType 拿不到结果按 null/异常处理	ContentResolver:798（用法见 : 929）

bash

```
# 发布/等待链现场
adb shell dumpsys activity providers
# 重点看：ContentProviderRecord{... connections=[...]} 的连接数与 stable/unstable 计数；
# "Launching" 段列出的就是此刻 mLaunchingProviders 里的条目。
# 客户端等待卡死时看 binder（system_server 侧不阻塞，等在客户端）
adb shell dumpsys activity processes | grep -A8 "<你的进程名>"
# oom adj=... state=... 里若见 PROCESS_STATE_HEAVY_WEIGHT / awaiting provider 线索，
# 再看 exit-info 里的 REASON_DEPENDENCY_DIED。
# 尸检：宿主反复死亡的记录
adb shell dumpsys activity exit-info <包名>
```

设计洞察 · 逐段精读：为什么只有 Provider 需要"发布"这个环节

Activity/Service/Broadcast 的宿主都不持"全局唯一资源句柄"，而 provider 的 authority 是**全局命名空间里的单例**——必须有一个"何时对外可见"的明确时刻，且发布前所有等待者必须可枚举（否则唤醒就是广播式的瞎猜）。mLaunchingProviders/notifyAll/逐连接通知三件套，本质是把"单例资源的可见性协议"显式建模——这也是为什么四大组件里只有 Provider 有同步等待与超时杀宿主的双重兜底（第六章"为什么 Provider 同步等待而其他组件异步"的机制版答案）。

源码锚点 · 第十三章锚点速查

ContentProviderHelper.java:100 mLaunchingProviders / :431-521 孵化决策与记账 / :549-570 外部调用方 cpr.wait / :636 publishContentProviders
:1357 incProviderCountLocked / :1413 decProviderCountLocked（5s 延迟） / :1695 removeDyingProviderLocked
ActivityThread.java:7016/1888 acquireProvider 等待 / notifyContentProviderPublishStatus / ContentProviderRecord.java:217 onProviderPublishStatusLocked
ContentResolver.java:786/794 PUBLISH 10s / READY 10s+10s

第十四章 updateOomAdjLSP 内部：一次全量核算的流水线与空进程回收

设计洞察 · 学习目标

1. 画出 updateOomAdjLocked → updateOomAdjLSP → performUpdateOomAdjLSP → updateOomAdjInnerLSP 的调用拓扑并解释 pending targets 合批；2. 背出 updateOomAdjInnerLSP 的五段式结构；3. 手推 assignCachedAdjIfNecessary 的 900~999 分档算法；4. 列全 updateAndTrimProcessLSP 的四种 kill 判决并说出每种对应的 ApplicationExitInfo subreason

第六章从"输入因子"视角讲了 `computeOomAdj`（6.4），本章下探一层：**一次全量核算的流水线本身**——合批、计算、缓存分档、环重试、回收判决、内核落账。这是 AMS 里最精巧的一段基础设施，也是"为什么一个进程的优先级变了，别的进程也跟着动"的答案。

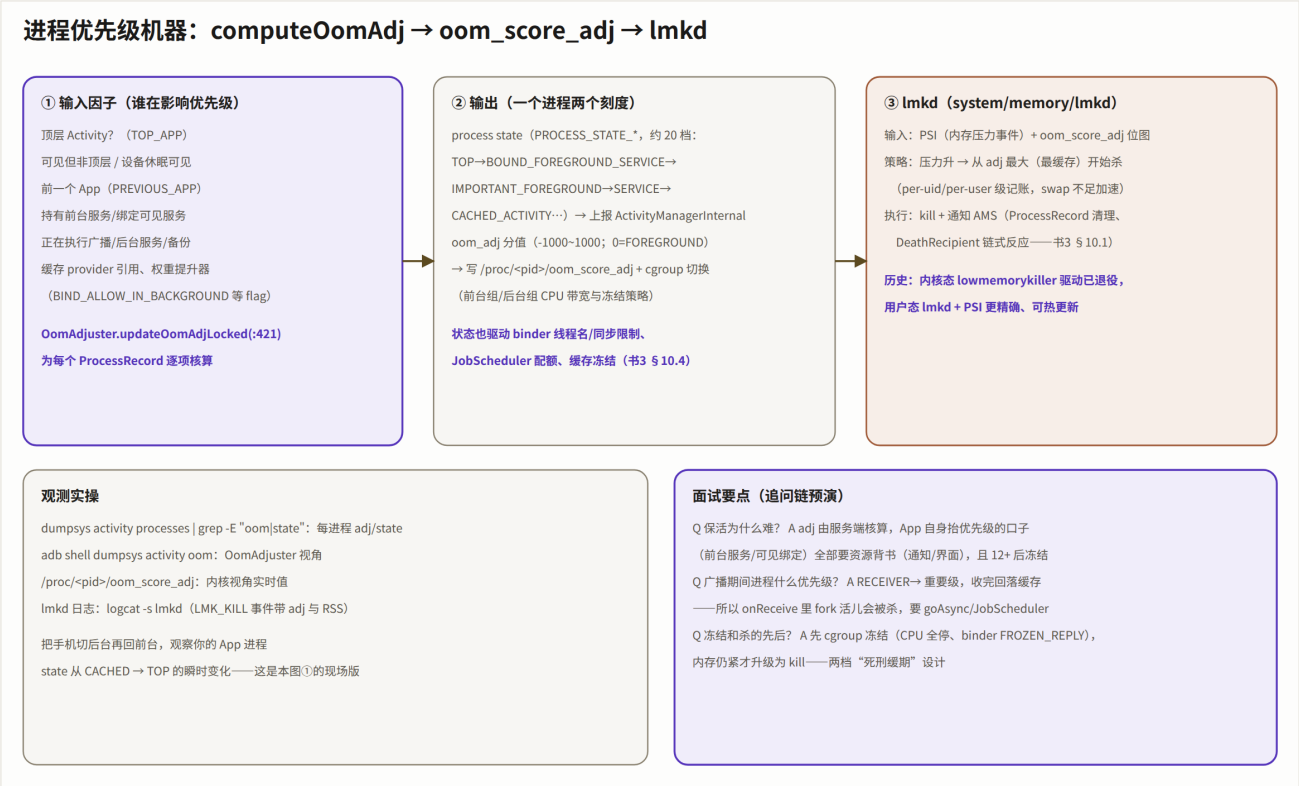


图 5 进程优先级机器：输入因子→process state/oom_adj→cgroup/lmkd，含观测与面试追问链

14.1 调用拓扑：合批优先于全量

java frameworks/base/services/core/java/com/android/server/am/OomAdjuster.java:421-447（节选+讲解注释）

```
void updateOomAdjLocked(String oomAdjReason) {
synchronized (mProcLock) {
updateOomAdjLSP(oomAdjReason); // 讲解注释①：全量更新的对外入口。
}
}
```

```
private void updateOomAdjLSP(String oomAdjReason) {
if (checkAndEnqueueOomAdjTargetLocked(null)) {
// 讲解注释②：**已有一轮更新在跑** → 本请求不重复全量，只把自己
// （null=全部）塞进 pending 集合，等在跑的那轮收尾时补算。
return;
}
try {
mOomAdjUpdateOngoing = true;
performUpdateOomAdjLSP(oomAdjReason);
} finally {
// 讲解注释③：本轮期间被塞进 pending 的目标统一补一轮——
// "窗口期内的 N 次请求合并为 1+N 次精确补算"。
mOomAdjUpdateOngoing = false;
updateOomAdjPendingTargetsLocked(oomAdjReason);
}
}
```

逐段精读：②③是**写合并（write coalescing）**——OomAdjuster 是 system_server 最贵的基础设施之一（全量要扫几百个进程），但它的触发源极多（广播、服务、Activity、绑定……）。合批把"每个事件各跑一轮全量"压成"一轮全量 + 窗口内增量补算"，这是高频重算系统的标准生存术，与第四章的"广播防重入闹钟"（mBroadcastsScheduled）同属一族。

14.2 updateOomAdjInnerLSP：五段式流水线

java frameworks/base/services/core/java/com/android/server/am/OomAdjuster.java:744-880（节选+讲解注释）

```
private void updateOomAdjInnerLSP(String oomAdjReason, final ProcessRecord topApp,
ArrayList<ProcessRecord> processes, ActiveUids uids, boolean potentialCycles,
boolean startProfiling) {
...
final boolean fullUpdate = processes == null;
// 讲解注释①：第一段·准备：全量时直接拿 LRU 链表（mProcessList.
// getLruProcessesLOSP——AMS 的进程总花名册，按最近使用排序）；
// uid 记录先 reset（本轮 uid 级聚合从零重算）。
ArrayList<ProcessRecord> activeProcesses = fullUpdate ? mProcessList.getLruProcessesLOSP()
: processes;
...
// 讲解注释②：第二段·重置：每个进程的可达位、环标记、原始 adj
// 全部清零（adjSeq 不等于本轮的进程才重置——被上一轮刚算过的跳过）。
for (int i = numProc - 1; i >= 0; i--) {
...
state.setReachable(false);
if (state.getAdjSeq() != mAdjSeq) {
state.setContainsCycle(false);
state.setCurRawProcState(PROCESS_STATE_CACHED_EMPTY);
state.setCurRawAdj(ProcessList.UNKNOWN_ADJ);
...
}
}
// 讲解注释③：第三段·计算：**从 LRU 链尾（最不重要）向链首**逐个
// computeOomAdjLSP——从尾向头，让"被客户端抬高"的宿主进程
// 后算，天然吸收客户端传来的提升（绑定传播方向）。
for (int i = numProc - 1; i >= 0; i--) {
...
```

```
computeOomAdjLSP(app, ProcessList.UNKNOWN_ADJ, topApp, fullUpdate, now, false,
computeClients);
retryCycles |= state.containsCycle();
state.setCompletedAdjSeq(mAdjSeq);
}
// 讲解注释④：第四段·缓存分档 + 重排：CacheOomRanker 可选地把
// 纯缓存进程按"更聪明的信号"（启动频率、存储占用等）重排 LRU，
// 再 assignCachedAdjIfNecessary 分配 900~999 的档位（14.3）。
if (mCacheOomRanker.useOomReranking()) {
mCacheOomRanker.reRankLruCachedAppsLSP(mProcessList.getLruProcessesLSP(),
mProcessList.getLruProcessServiceStartLOSP());
}
assignCachedAdjIfNecessary(mProcessList.getLruProcessesLOSP());
// 讲解注释⑤：第五段·环重试：绑定图有环（A 绑 B、B 绑 A）时，
// 单遍计算可能不收敛——最多重试 10 轮，直到没有进程被提升。
int cycleCount = 0;
while (retryCycles && cycleCount < 10) {
cycleCount++;
retryCycles = false;
...
}
...
// 讲解注释⑥：第六段·应用与回收：逐进程 applyOomAdjLSP 落账，并做
// 空进程/超容回收判决（14.4）；随后 PSS 采集计划、uid 聚合上报。
boolean allChanged = updateAndTrimProcessLSP(now, nowElapsed, oldTime, activeUids);
...
}
```

逐段精读：③的**遍历方向**值得单独背下——"从链尾向链首"不是随手写的：LRU 链尾是最缓存的进程，它们往往是绑定图的叶子；先算叶子，宿主再算时就能在单遍内看到"客户端拉抬"，把多遍收敛的需求压到只剩真环。⑤的 10 次上限则是"理论不保证收敛，工程上必须收敛"的务实注脚。

14.3 assignCachedAdjIfNecessary：900~999 的分档算法

java frameworks/base/services/core/java/com/android/server/am/OomAdjuster.java:902-955（节选+讲解注释）

```
int curCachedAdj = ProcessList.CACHED_APP_MIN_ADJ; // 900
int nextCachedAdj = curCachedAdj + (ProcessList.CACHED_APP_IMPORTANCE_LEVELS * 2); // 910
int curCachedImpAdj = 0;
int curEmptyAdj = ProcessList.CACHED_APP_MIN_ADJ + ProcessList.CACHED_APP_IMPORTANCE_LEVELS;
...
// 讲解注释①：空进程槽 = 缓存总容的一半（computeEmptyProcessLimit
// = total/2，ActivityManagerConstants:1113）：32 容 → 空 16 槽。
final int emptyProcessLimit = mConstants.CUR_MAX_EMPTY_PROCESSES;
final int cachedProcessLimit = mConstants.CUR_MAX_CACHED_PROCESSES
- emptyProcessLimit;
...
// 讲解注释②：缓存档与空档**共用 900~999 的号码段但分层排队**：
// 活动缓存从 900 起步、空进程从 905 起步，每档步进 10
// （CACHED_APP_IMPORTANCE_LEVELS=5 × 2）。
for (int i = numLru - 1; i >= 0; i--) {
...
switch (state.getCurProcState()) {
case PROCESS_STATE_CACHED_ACTIVITY:
```

```
case ActivityManager.PROCESS_STATE_CACHED_ACTIVITY_CLIENT:
case ActivityManager.PROCESS_STATE_CACHED_RECENT:
// 讲解注释③：按 LRU 顺序消费档位：curCachedAdj 每满
// factor 个进程前进一步；连接组（connectionGroup，
// 同 App 的关联进程）可共享档位不占名额。
...
case ActivityManager.PROCESS_STATE_CACHED_EMPTY:
// 讲解注释④：空进程同理消费空档，超出槽位的直接
// 给 CACHED_APP_MAX_ADJ（999）——最先被 lmkd 盯上。
...
}
}
```

逐段精读：**"缓存"与"空"不是形容词，是两个资源池**——cached（还有 Activity 骨架，冷启动能省一半）与 empty（什么都不剩，杀了不心疼）。把总容 32 对半分成两个池，就是"宁可少缓存几个 Activity，也要给空进程留位置"的取舍：空进程是应用的"保温杯"（省去 fork+attach 的固定成本），而缓存 Activity 是"回放录像"（省去全部冷启动）——两者价值不同，不能混在一个淘汰队列里按 LRU 硬排。

14.4 updateAndTrimProcessLSP：回收判决书

java frameworks/base/services/core/java/com/android/server/am/OomAdjuster.java:1037-1145（节选+讲解注释）

```
private boolean updateAndTrimProcessLSP(final long now, final long nowElapsed,
final long oldTime, final ActiveUids activeUids) {
...
// 讲解注释①：回收总开关：shouldKillExcessiveProcesses——低内存
// 检测器说"不缺"时，超容杀全部豁免（只算数、不动刀）。
final boolean doKillExcessiveProcesses = shouldKillExcessiveProcesses(now);
...
final int emptyProcessLimit = doKillExcessiveProcesses
? mConstants.CUR_MAX_EMPTY_PROCESSES : Integer.MAX_VALUE;
final int cachedProcessLimit = doKillExcessiveProcesses
? (mConstants.CUR_MAX_CACHED_PROCESSES - emptyProcessLimit) : Integer.MAX_VALUE;
...
for (int i = numLru - 1; i >= 0; i--) {
ProcessRecord app = lruList.get(i);
...
if (!app.isKilledByAm() && app.getThread() != null) {
if (state.getCompletedAdjSeq() == mAdjSeq) {
// 讲解注释②：把第三段算好的账**落地**：写 /proc 上的
// oom_score_adj（经 lmkd socket，14.5）、切 cgroup、
// 冻结/解冻——第六章 6.1 说"三件事"的执行现场。
applyOomAdjLSP(app, true, now, nowElapsed);
}
switch (state.getCurProcState()) {
case PROCESS_STATE_CACHED_ACTIVITY:
case ActivityManager.PROCESS_STATE_CACHED_ACTIVITY_CLIENT:
mNumCachedHiddenProcs++;
numCached++;
...
if ((numCached - numCachedExtraGroup) > cachedProcessLimit) {
// 讲解注释③：判决一：缓存超容——最旧的先杀。
app.killLocked("cached #" + numCached,
```

```
"too many cached",
ApplicationExitInfo.REASON_OTHER,
ApplicationExitInfo.SUBREASON_TOO_MANY_CACHED, true);
}
break;
case PROCESS_STATE_CACHED_EMPTY:
if (numEmpty > mConstants.CUR_TRIM_EMPTY_PROCESSES
&& app.getLastActivityTime() < oldTime) {
// 讲解注释④：判决二：空进程超过 trim 线（16/2=8）
// 且闲置超 30 分钟（mMaxEmptyTimeMillis）→ "空太
// 久"杀。注意：这是**主动回收**，与 lmkd 的内存
// 压力杀（第七章 7.3）互补——一个按龄、一个按压。
app.killLocked("empty for " + ((now
- app.getLastActivityTime()) / 1000) + "s",
"empty for too long",
ApplicationExitInfo.REASON_OTHER,
ApplicationExitInfo.SUBREASON_TRIM_EMPTY, true);
} else {
numEmpty++;
if (numEmpty > emptyProcessLimit) {
// 讲解注释⑤：判决三：空进程超容 → 立杀。
app.killLocked("empty #" + numEmpty,
"too many empty",
ApplicationExitInfo.REASON_OTHER,
ApplicationExitInfo.SUBREASON_TOO_MANY_EMPTY, true);
}
}
break;
default:
mNumNonCachedProcs++;
break;
}
if (app.isolated && psr.numberOfRunningServices() <= 0
&& app.getIsolatedEntryPoint() == null) {
// 讲解注释⑥：判决四：isolated 进程无用武之地即杀——
// 它天生不可复用（每次孵化都是全新进程），留着纯烧内存。
app.killLocked("isolated not needed", ApplicationExitInfo.REASON_OTHER,
ApplicationExitInfo.SUBREASON_ISOLATED_NOT_NEEDED, true);
} else {
updateAppUidRecLSP(app);
}
...
}
}
return mService.mAppProfiler.updateLowMemStateLSP(numCached, numEmpty, numTrimming);
}
```

逐段精读：③④⑤⑥四张判决书全部带着 `ApplicationExitInfo` 的 subreason——**每一次主动回收都留了尸检字段**。用 `dumpsys activity exit-info` 或 `logcat -s am_kill` 能看到 "too many cached"/"empty for 1800s"/"isolated not needed" 的原文，把"谁杀的、为什么杀"从猜测变成查询。②与⑥之间还有一条隐含的纪律：**apply 在 count 之前**——先落账再判决，保证 lmkd 手里的 adj 位图始终是最新账本，避免"杀了一个已经该升档的进程"。

14.5 applyOomAdjLSP 落账：从内存数字到内核

`applyOomAdjLSP`（`:2550`）把一次核算翻译成四路落账：**adj**——`ProcessList.setOomAdj(pid, uid, adj)`（`:2587`）经 socket 发 `LMK_PROCPRIO` 命令给 lmkd（`ProcessList.java:1390-1408`，带 250ms 慢日志 "SLOW OOM ADJ"）；**调度组**——SCHED_GROUP_TOP_APP/DEFAULT/BACKGROUND 映射到 cputl cgroup，TOP 的 UI/Render 线程还单独 setThreadPriority（`:2680` 一带）；**冻结**——`updateAppFreezeStateLSP`（`:2708`）接 CachedAppOptimizer（第七章 7.4）；**状态上报**——`setProcessState` 回调 App、`maybeUpdateUsageStatsLSP` 喂使用统计。四路落账只发生在"真的变了"时（`procStatesDifferForMem` 等判定）——**写放大的控制与计算本身同等重要**，这是它 500 行代码里一半是 diff 检查的原因。

14.6 adj 常量全表（ProcessList.java:188-264）

adj 常量	值	含义
FOREGROUND_APP_ADJ	0	前台（可见可交互）
PERCEPTIBLE_RECENT_FOREGROUND_APP_ADJ	50	最近前台的感知进程
VISIBLE_APP_ADJ	100	可见非顶层（另按层级加 0~99）
PERCEPTIBLE_APP_ADJ	200	可感知（音乐/导航类前台服务）
PERCEPTIBLE_LOW_APP_ADJ	250	低档可感知
BACKUP_APP_ADJ	300	正在备份
SERVICE_ADJ	500	服务进程（A 类）
HOME_APP_ADJ	600	桌面
PREVIOUS_APP_ADJ	700	上一应用
SERVICE_B_ADJ	800	服务进程 B 类（老旧降级）
CACHED_APP_MIN_ADJ	900	缓存池下沿
CACHED_APP_MAX_ADJ	999	缓存池上沿（最先杀）
UNKNOWN_ADJ	1001	尚未核算（临时占位）
PERSISTENT_SERVICE_ADJ	-700	持久服务宿主

> **设计洞察 · 逐段精读：为什么 negative 区间（-700~0）系统自己人独占**
>
> lmkd 只管 0 以上的"平民区"；负 adj 是系统进程与持久服务的**免杀区**——lmkd 的扫描永远不会碰它们。把"绝对不可杀"做成编号而非开关，让内核/lmkd 的选择逻辑保持单一（沿位图从 999 往下找受害者即可），不需要任何特判。**用数值区间表达权限边界**，与第四章用队列隔离前台后台是同一种设计直觉：让不变量长在数据结构里。

源码锚点 · 第十四章锚点速查

OomAdjuster.java:421/428/744/902/1037/2550 五级调用拓扑与五段式 / :2587 setOomAdj 落账 / :2708 冻结钩子
ProcessList.java:188-264 adj 常量表 / :1390-1408 setOomAdj→LMK_PROCPRIO socket / :202 档距 5
ActivityManagerConstants.java:148 缓存 32 / :1113 空池=总容一半 / :757 空进程 30min / :746 trim 线

附录 A 观测实操

目标	命令	对应机制
启动全链	am start -W -n pkg/Activity	WaitTime/TotalTime 分段
组件全景	dumpsys activity activities / services / broadcasts / providers	四组件各自的 dump
进程优先级	dumpsys activity processes + /proc/<pid>/oom_score_adj	computeOomAdj 现场
ANR 现场	/data/anr/ + dropbox	AnrHelper.dumpStackTraces
广播队列	dumpsys activity broadcasts	BroadcastQueue 内景
杀进程链	logcat -s lmkd am_kill	lmkd→AMS 清账

附录 B 自检题

- 默画冷启动十步，并标注每步所在进程。
- ClientTransaction 的 callbacks 与 lifecycleStateRequest 各承担什么？cycleToPath 补的是什么？
- 四组件的"欠条-兑现"分别叫什么名字（四个 pending 容器）？
- computeOomAdj 的输入因子列五个；前台服务把进程抬到哪一档？
- 有序广播 ANR 的传导链是什么？goAsync 改变了链条中的哪一环？

附录 C 崩溃/卡顿与调度的对照表

症状	调度层解释	排查入口
ANR Input（5s）	主线程没回执输入事件（书5 §7）	traces 主线程栈
ANR Broadcast（10s/60s）	onReceive 未 finish	BroadcastQueue 超时接收者

症状	调度层解释	排查入口
ANR Service（20s/200s）	onCreate/onStartCommand 超时	ActiveServices 计时
ANR Provider（10s）	宿主未及时 publish	ContentProviderHelper 计时
IllegalStateException: Not allowed to start service	缓存态后台 startService 熔断	改 startForegroundService
ForegroundServiceDidNotStartInTimeException	5s 内未 startForeground	FGS 通知链
Activity 泄漏	静态持有 Activity 引用，但调度层早已 destroy	leaks 工具+DestroyActivityItem 路径
重复 onCreate 而非 onNewIntent	launchMode/flags 判决为 standard	§1.5 裁决顺序表

附录 D 快问快答（面试扫雷 20 条）

AMS 与 ATMS 的分界？——进程调度（am/）vs 任务与窗口（wm/），2019 拆分。 冷/暖/热启动定义？——进程新建/已存在新 Activity/仅 resume；am start -W 的 LaunchState 直接标出。 - onNewIntent 触发条件？——launchMode 判决复用实例（singleTop 栈顶等）。 - ActivityRecord 与 ActivityClientRecord？——系统侧/App 侧镜像，token 为桥。 - ClientTransaction 一次能带几个回调？——多个（callbacks 列表），终态一个。 - 为什么 Application.onCreate 可能晚于首个 Activity 的部分初始化？——makeApplicationInner 在 performLaunchActivity 内触发。 Service onBind 返回 null 会怎样？——绑定成功但收不到 binder（onServiceConnected 拿 null）。 stopService 与 unbindService 组合语义？——started/bound 两本账独立，双清才销毁。 广播防自启限流？——后台 30s/30 次（ActiveServices/AMS 双限流）。

有序广播能截获吗？——abortBroadcast 仅 ordered 链有效。 goAsync 的正确用法？——PendingResult.finish() 必须在超时线内调。 ContentProvider 的 multi-process 声明？——android:multiprocess 极少用（进程内直连，失去统一账本）。 query 大表为什么不卡 Binder？——CursorWindow 共享内存分页。 oom_adj 与 nice 的区别？——内存查杀依据 vs CPU 调度优先级（两套体系，cgroup 才是 CPU 待遇）。

缓存进程多少档？——CACHED_ACTIVITY_MIN~MAX 分组限容，LRU 逐档淘汰。 冻结中的 App 能收广播吗？——默认 deferred，解冻补投。 LMK 与 lmkd？——内核驱动退役，用户态守护 + PSI。 ANR 弹窗谁弹？——AMS 侧 showAnrUi 经权限校验拉起系统对话框进程。 am kill 与 kill -9 差别？——走 AMS 清账（组件/连接/notification 同步回收）。

学完本册下一步？——第五册（窗口与输入）承接"首帧之后"，第六册（PMS）承接"包从哪来"。

教科书习题集（分级）

用法建议：第一级闭卷手写（检验记忆），第二级对着源码作答（检验机制），第三级限时 30 分钟写方案（检验迁移）。参考答案要点只给"判分锚点"，展开论证才是得分主体。

第一级：简答题（10 题，检验概念理解）

1. **冷启动十步中发生了几次 Binder 跳、几次 socket 跳？各发生在哪两步之间？**

参考答案要点：三次 Binder 跳（①App→ATMS、⑤App 回报 attachApplication、⑥system_server→App 下发事务）+ 一次 socket 跳（④system_server→zygote fork）；首帧渲染还有 App→WMS/SF 的第三次跨进程（第十章口径按三段计）。

2. **ClientTransaction 的 callbacks 与 lifecycleStateRequest 各承担什么？cycleToPath 补的是什么？**

参考答案要点：callbacks 描述"发生什么"（LaunchActivityItem 等，可多个），lifecycleStateRequest 描述"终到哪"（终态一个）；cycleToPath 沿状态机补齐当前态到目标态的中间事件（第二章 2.1③）。

3. **四组件各自的"欠条-兑现"容器分别叫什么？在哪个函数里统一兑现？**

参考答案要点：Activity 的 pending 事务、Service 的 mPendingServices、Broadcast 的 mPendingBroadcast、Provider 的 mLaunchingProviders；统一兑现点是 attachApplicationLocked（第一章 1.6）。

4. **Service 的 20s 与 200s 超时由什么决定？计时的起点字段是什么？**

参考答案要点：shouldExecServicesFg()（executing 服务时是否前台档）决定；起点是 bumpServiceExecutingLocked 记下的 r.executingStart，从栈顶找最老的未归栈回调比对（第十一章 11.6）。

5. **startForegroundService 逾期未调 startForeground 的完整追责链是什么？**

参考答案要点：sendServiceArgsLocked 上弦 30s（mServiceStartForegroundTimeoutMs）→ serviceForegroundTimeout 先 stopService，再延迟 10s 走 ANR（"did not then call Service.startForeground()"）；若期间服务被销毁则直接抛 ForegroundServiceDidNotStartInTimeException（第十一章 11.6）。

6. **动态注册的接收者回调一定在哪个线程？机制根源是哪一行？**

参考答案要点：注册时绑定的 Handler 线程（默认主线程）；根源是 ReceiverDispatcher.performReceive 里的 mActivityThread.post(args.getRunnable())（LoadedApk:1879，第十二章 12.3）。

7. **有序广播超时的计时单位是"单条广播"还是"单个接收者"？两条队列的超时线各是多少？**

参考答案要点：单个接收者接力计时（receiverTime + mConstants.TIMEOUT，每条队列 10s/60s 由 BROADCAST_FG/BG_TIMEOUT 覆盖）；上一个慢接收者不吃掉下一个的窗口（第十二章 12.4）。

8. **T 上 App 调用方等 Provider 发布时，到底阻塞在哪里？谁唤醒它？**

参考答案要点：阻塞在客户端 ActivityThread.acquireProvider 的 ProviderKey.mLock.wait（AMS 只回 provider==null 的 holder）；唤醒来自 notifyContentProviderPublishStatus 回调（成功或失败都走它，第十三章 13.3）。

9. **stable 与 unstable Provider 引用在宿主死亡时的处置差别是什么？**

参考答案要点：stable 客户端被 killLocked（REASON_DEPENDENCY_DIED，"depends on provider ..."）；unstable 客户端只收 unstableProviderDied 回调自行善后（第十三章 13.6）。

10. **缓存池 32 容下，空进程的槽位、trim 线、闲置时限各是多少？**

参考答案要点：空槽 = 总容一半 = 16（computeEmptyProcessLimit）；trim 线 = 16/2 = 8（CUR_TRIM_EMPTY_PROCESSES）；闲置 30 分钟（mMaxEmptyTimeMillis）后"empty for too long"即杀（第十四章 14.4）。

第二级：分析题（7 题，检验机制理解）

11. **追踪调用链**：从 `Context.startService` 到 `onStartCommand`，画出完整调用图并标注：两次"欠条"挂账点、executingServices 压栈点、两台超时闹钟的上弦点。

参考答案要点：挂账点在 startServiceInnerLocked（pendingStarts:879）与 bringUpServiceLocked（mPendingServices:4301）；压栈在 realStartServiceLocked 的 bumpServiceExecutingLocked；闹钟上弦分别在压栈时（20s/200s）与 sendServiceArgsLocked:4478（30s）。

12. **失败推演**：若宿主进程在 publishContentProviders 之前被杀，等待方分别经历什么？区分 App 调用方、外部调用方、以及 mLaunchingProviders 里的记账。

参考答案要点：removeDyingProviderLocked：未超 MAX_RETRY_COUNT 时 cpr 留在 launching 清单等重启（后来者继续等）；waiting 连接在 always=false 时不动，超限则一律 status=false 通知——App 方醒来拿 null（"Failed to find provider info"），外部方被 cpr.notifyAll 唤醒返回 null（第十三章 13.6）。

13. **对比分析**：Service 的 serviceTimeout 与 Broadcast 的 broadcastTimeoutLocked 都有"isDebugging 豁免 + 判决后系统继续推进"的共性，请逐条对齐两个函数的结构，并指出 ANR 与"链继续"为何不矛盾。

参考答案要点：对齐项：调试豁免、迟到的闹钟无罪释放（执行栈空/队列空）、判决走 mAnrHelper、判决后调度继续；不矛盾因为 ANR 惩罚的是"单次调度步超时"的进程，而队列推进是系统自身的有限步终止性保证（第十一章 11.6、第十二章 12.5）。

14. **机制理解**：为什么 updateOomAdjInnerLSP 从 LRU 链尾向链首遍历 computeOomAdjLSP？若反过来会发生什么？环重试的 10 次上限兜住了什么？

参考答案要点：链尾是缓存叶子，先算叶子让宿主单遍吸收客户端拉抬；反序则每次都要等宿主先算完，传播退化成 O(深度) 轮；10 次上限保证绑定环（A↔B）最终收敛或放弃，避免死循环（第十四章 14.2）。

15. **失败推演**：把 `deliverToRegisteredReceiverLocked` 的 skip 逻辑与冻结机制串起来：一个被冻结进程的动态接收者会经历什么？解冻后呢？

参考答案要点：冻结进程的广播默认 deferred（CacheAppOptimizer/CachedAppOptimizer 路径），受限进程按 AppRestrictionController 策略跳过并记 DELIVERY_SKIPPED；解冻后冻结期积压广播补投——所以"冻结导致广播延迟但通常不丢失"（第十二章 12.6、第六章 6.2）。

16. **机制理解**：incProviderCount 的 updateLru 分支为什么只在客户端 adj ≤ PERCEPTIBLE_LOW_APP_ADJ 时把宿主"回炉"？decProviderCount 的 5 秒延迟回收消灭了什么抖动？

参考答案要点：可感知客户端的访问才值得给宿主 LRU 优待（provider 启动贵，但纯后台访问不该白拿待遇）；5s 延迟让"释放又立刻重取"不触发真拆建，消除连接对象与 OomAdj 的抖动（第十三章 13.5）。

17. **对比分析**：把 startForegroundService 的 tempAllowlistUidLocked（11.3⑨）与 ANR 追责（11.6）放在同一张时间线上，解释"先给能力、后追责"为什么不是漏洞。

参考答案要点：30s 豁免窗口是追责链的**前置条件**（没有执行能力就谈不上义务），且窗口与追责时钟同长同源（mServiceStartForegroundTimeoutMs）；到期即止损（stopService）+ 证据（ANR）双收，豁免从不外溢（第十一章 11.3/11.6）。

第三级：设计题（3 题，检验迁移能力）

18. **架构演进**：假设要为"可预取的 Service"设计新语义（系统在预测到启动意图时提前孵化进程但不回调 onCreate），你会复用本章哪些记账结构？欠条-兑现模型需要补什么状态？

参考答案要点：可复用 mPendingServices（预孵化即"已挂欠条未兑现"）+ ServiceRecord 增 preLaunched 状态位与撤销超时；关键补丁：兑现点（attachApplicationLocked）按预取位分流到"继续兑现"或"回滚销毁"，并给预取配额防滥孵。

19. **排障方案**：用户报告"应用退后台几分钟后被杀，音乐还在响时也不例外"。请设计分层定位方案（命令→现象→机制对应章节），并给出三种可能的根因排序。

参考答案要点：先 logcat -s lmkd am_kill 看击杀方与当时 adj；再 dumpsys activity processes 复现前后台切换抓 state/adj 变化；然后 exit-info 查 subreason（TRIM_EMPTY/TOO_MANY/DEPENDENCY_DIED 分流）。根因排序：前台服务未真正升级（通知/类型问题，adj 没到位）→ 稳定 Provider 宿主死亡连坐 → 空池 trim 误伤（后两者对应第十三/十四章）。

20. **架构演进**：如果把有序广播的"接力超时"改为"全链总超时"，会改善什么、破坏什么？给出你的结论与一个折中设计。

参考答案要点：改善：总时长可预测，链尾延迟有上界；破坏：责任隔离失效（慢接收者吃掉后续窗口，冤案扩散）、N 个接收者的公平性坍塌。折中：保留接力超时 + 对整链加一个宽松的统计性"慢链"标记（类似 SLOW_TIME 的 deferral 思路），只降调度优先级不 ANR（第十二章 12.4/12.5）。

术语表（按拼音排序）

术语	解释
ANR（Application Not Responding）	应用无响应：输入 5s/前台广播 10s/后台广播 60s/前台服务 20s/后台服务 200s 五源超时的统一判罚，出口统一走 AnrHelper（§6.3）
Binder 代理（IApplicationThread）	AMS 持有的目标进程调度通道：四组件事务的"收件人"，attachApplication 后才就位（§1.6）
BroadcastFilter	动态接收者在 AMS 侧的过滤登记：IntentFilter + 权限 + ReceiverList 归属（§12.2）
BroadcastRecord	一次广播发送的总账：receivers 名单、各接收者投递状态（DELIVERY_*）、超时/结果追踪（§12.5）
BroadcastQueue	广播队列：T 上前台/后台 + 两条 offload 共四条，超时线与延迟策略按队列配置（§12.4）
CacheOomRanker	缓存进程重排器：用启动频率等信号把纯缓存进程的 LRU 位次重排后再分档（§14.2④）
cgroup（控制组）	内核资源分组：adj 定生死、cgroup 定待遇（CPU 带宽/冻结资格），由 applyOomAdjLSP 落账（§6.2、§14.5）
ClientTransaction	生命周期事务：callbacks（发生什么）+ 终态请求（到哪去），App 侧 TransactionExecutor 补路径执行（第二章）
ContentProviderConnection	provider 连接账：stable/unstable 双计数 + waiting 位，是宿主保活待遇的记账单位（§13.5）
ContentProviderRecord	provider 单例账：provider binder（null 即未发布）、launchingApp、connections、死亡重试计数（§13.1）
fgRequired（前台服务义务）	startForegroundService 留下的"必须 30s 内 startForeground"追踪位，违约先停服务再 ANR（§11.6）
lmkd	用户态内存查杀守护：持 adj 位图与 PSI 压力事件，"AMS 出价、内核报时、lmkd 落槌"（§7.3）
mLaunchingProviders	全局"发布中"清单：provider 孵化去重表 + 等待者枚举的依据（§13.2）
mPendingServices	待兑现服务清单：进程孵化期间挂账的 ServiceRecord，attachApplication 时补发（§3.1、§11.3）

术语	解释
oom_score_adj	写给内核/lmkd 的查杀序数：0（前台）到 999（缓存上沿），经 LMK_PROCPRIO socket 同步（§14.5）
PendingResult	goAsync 拿到的延时回执单（ReceiverDispatcher.Args 的基类）：finish() 必须在超时线内调用（§12.3）
ProcessRecord	进程总账：pid、IApplicationThread、LRU 位次与 mState/mServices/mReceivers/mProviders 各分账的宿主
process state（进程状态机）	PROCESS_STATE_* 双刻度之一：给框架消费（配额/冻结/限制），TOP 到 CACHED_ACTIVITY_MIN 分档（§6.4）
ReceiverDispatcher	动态注册桥：持 IIntentReceiver 桩 + 注册线程 Handler，post 与代收回执的执行者（§12.3）
ServiceRecord	服务本体账：pendingStarts（onStartCommand 欠条）、bindings、fgRequired、executingStart（§11.1）
超时线	ANR 家族的计时口径：永远按"单次调度步"（一次回调/一个接收者/一次事件）起算而非整个业务流程（§12.5）
冻结（freezer）	缓存进程进 freezer cgroup：CPU 全停、binder 同步速败、广播绕行——"可逆的杀"（§6.2、§7.4）
缓存进程	cached（含 Activity 骨架）与 empty（全空）两个资源池：总容 32 对半分，分档 900~999 按档距 10 排（§14.3）
欠条-兑现	AMS 对"目标进程不存在"的标准答案：记账（pending 容器）后等孵化回调集中兑现（§1.6）
引用计数	incProviderCount/decProviderCount 的 stable/unstable 双账：决定宿主死亡时客户端"陪葬还是善后"（§13.5）
有序广播	按优先级串行投递、finishReceiver 后才走下一位的广播：超时按单接收者接力计时（§12.4）
粘性广播	留在 mStickyBroadcasts 供后注册者查询的广播：T 上机制仍在、已弃用，收紧点在注册导出标志（§12.2）

排障手册：AMS 与四大组件调度问题定位

通用套路：**先问"谁在等谁"（dumpsys 看账本）→ 再看"谁卡在哪"（traces 看现场）→ 最后对账（源码/本册章节）**。以下六个场景覆盖本册四组件与优先级两大主题，命令均可在 userdebug 版直接执行。

场景 1：ANR——executing service（Service 超时）

症状：弹窗"应用无响应"；logcat 出现 ANR in <包名>... Reason: executing service <Service 类名>；`/data/anr/` 落了 traces。

定位命令：

```bash
adb logcat -d | grep -E "am_anr|ANR in|executing service"
adb pull /data/anr/ traces_dir/
# traces 中找 main 线程：若停在 onCreate/onStartCommand 的业务栈 → 回调超时实锤；
# 若 main 空闲而 ANR 仍在 → 查 executingServices 是否被 bump 后未回执（回执丢失）。
adb shell dumpsys activity services <包名>
# 看 ServiceRecord 的 createTime/startRequested/executingStart 与 app= 进程归属。
```

根因：onCreate/onStartCommand 在主线程做了重活（IO/锁/Binder 同步），20s（前台）或 200s（后台）内没有 serviceDoneExecuting 回执——executingStart 到点未出栈（§11.6 第一台闹钟）。

修复：重活移工作线程（onCreate 只做装配）；若必须同步初始化，检查是否被跨进程锁间接拖住（traces 找 "waiting to lock" 的锁持有者）；确认没有"回执路径被自己阻塞"的写法（在 onCreate 里同步 sendBroadcast 并等结果即自锁）。

场景 2：ANR——Broadcast of Intent（广播超时）

症状：`ANR ... Reason: Broadcast of Intent { act=... }`；`dumpsys activity broadcasts` 中该 BroadcastRecord 的 `anrCount>0`，超时 receiver 的 delivery 状态为 DELIVERY_TIMEOUT。

定位命令：

```bash
adb shell dumpsys activity broadcasts
# 找 "Timeout of broadcast" 日志行与 history 表：定位 receiver 与队列（foreground/background）。
adb logcat -d | grep -E "Timeout of broadcast|Receiver during timeout"
adb shell am monitor -p <包名> # 复现时观察接收者进程状态
```

根因：有序广播的某个接收者 10s/60s 内没回执 finishReceiver——主线程 onReceive 卡住，或 goAsync 的 PendingResult.finish() 忘调/晚调；注意**广播链本身已跳过它继续推进**，ANR 只属于卡住的那一环（§12.5⑧）。

修复：onReceive 里只做路由不做活；goAsync 场景把 finish() 绑进工作线程的 finally；manifest 接收者检查进程冷启动耗时（10s 里含孵化+attach+Application.onCreate，§12.1 第 6 步）。

场景 3：Provider 等待卡死 / "Failed to find provider info"

症状：调用 query 后业务线程长时间无响应（最长 20s：就绪超时 10s+10s），或直接抛 `IllegalArgumentException: Failed to find provider info for <authority>`；偶发目标进程被杀日志 "timeout waiting for provider"。

定位命令：

```bash
```

```
adb shell dumpsys activity providers
# 看 authority 是否在列表、"Launching" 段有无该条目（卡在 mLaunchingProviders）。
adb shell dumpsys activity exit-info <宿主包名>
# 找 REASON_DEPENDENCY_DIED："depends on provider <authority> in dying proc ..."。
adb shell ps -A | grep <宿主进程> # 宿主到底活没活着
```

根因：三类——宿主进程没起来（孵化失败/被 force-stop）；宿主起了但 publish 慢于 10s（Application.onCreate 太重，§13.4⑦之前超时）；稳定引用的宿主死亡把客户端连坐（§13.6⑥）。客户端侧等待位置是 ProviderKey.mLock.wait，不占 system_server（§13.3）。

修复：宿主瘦身（publish 前不做重活）；客户端对 unstable 语义改用 acquire/unstableProvider 并处理 unstableProviderDied 重试；检查是否误用稳定引用长时间持有跨进程 provider（连坐风险）。

场景 4：startForegroundService 抛异常或 ANR

症状：`ForegroundServiceDidNotStartInTimeException`（进程内 crash），或 ANR "Context.startForegroundService() did not then call Service.startForeground()"。

定位命令：

bash

```
adb logcat -d | grep -E "ForegroundServiceDidNotStartInTime|did not then call"
adb shell dumpsys activity services <包名> | grep -A6 "fgRequired"
# ServiceRecord 里 fgRequired=true 且 fgWaiting=true → 义务闹钟正挂着。
adb shell dumpsys activity processes | grep -B2 -A8 <包名>
# 确认进程当时是否处在缓存/受限档（临时豁免只覆盖 30s）。
```

根因：30s 窗口（mServiceStartForegroundTimeoutMs）内没调 startForeground：常见于 onStartCommand 排队等主线程、或服务被提前 stop 触发销毁路径的 crash（§11.6 第二台闹钟与第三行表格）。

修复：onCreate/onStartCommand 里第一件事就是 startForeground（用 startForegroundService 的那次 start 的 id 区分路径）；主线程先让路再干重活；审查 stopService 与 startForegroundService 的竞态（§11.4⑩的 delayedStop 补刀路径）。

场景 5：bindService 后 onServiceConnected 不回调

症状：bindService 返回 true 但回调迟迟不来；或回调拿到 null binder；或解绑后服务不销毁（双账本没清干净）。

定位命令：

bash

```
adb shell dumpsys activity services <包名>
# 看 ServiceRecord 的 bindings/AppBindRecord：requested/hasBound 状态与 connections 计数；
# onCreate 未执行则看是否卡在 realStartServiceLocked 之前（进程在不在、mPendingServices）。
adb shell dumpsys activity processes | grep -A5 <服务进程>
adb logcat -d | grep -E "scheduleBindService|publishService|deadObject"
```

根因：四选一——目标进程在孵化排队（欠条未兑现，§11.3⑧）；onBind 已回调但 publishService 未回传（binder 死亡，§3.2）；服务 processName 指向的进程被隔离限制；onBind 返回 null（绑定"成功"但无 binder 可给）。

修复：先确认 onCreate 是否已到（dumpsys 的 createTime）；onBind 侧检查返回对象是否跨进程可序列化；多进程 App 检查 android:process 拼写；死亡重绑依赖 ServiceConnection.onServiceDisconnected 后的重试逻辑（系统不自动重 bind）。

场景 6：进程优先级异常（该高不高 / 被提前杀）

症状：前台服务在跑但进程仍被 lmkd 杀；或退后台立即被杀而同类 App 不会；`dumpsys activity processes` 里 adj 与预期档位不符。

定位命令：

```bash
adb shell dumpsys activity processes | grep -B2 -A10 <包名>
# 对着 adj 常量表（§14.6）读档位：Perceptible=200、ForegroundService 档、cached 900+。
adb shell cat /proc/<pid>/oom_score_adj # 内核侧实际值（与 dumpsys 对账）
adb logcat -s lmkd # 击杀记录：adj、内存水位
adb shell dumpsys activity oom adj # 每个进程的 adjType（§6.4 因子命中点）
```

根因：前台服务没生效多为通知未就绪（startForeground 没调或类型不符，回退普通服务档）；或 OOM_ADJ 未及时重算（看 adjType 是否还是旧因子）；另查稳定 Provider 连坐（§13.6）与空池主动回收（§14.4，exit-info 的 subreason 可区分"被杀"与"被裁"）。

修复：按 dumpsys 的 adjType 对号入座——它就是 computeOomAdj 命中的因子名；确认 foregroundServiceType 与通知渠道；缓存态预期被裁属正常机制（30 分钟/空池线），优化方向是缩短重生路径而非对抗回收。

→ 深入：本册各章